

**BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

***Đề tài : “Xây dựng ứng dụng AI nhận
biết thực phẩm và tư vấn dinh dưỡng”***

Người hướng dẫn : Tiến sĩ Nguyễn Hồng Sơn

Sinh viên thực hiện : Phạm Thanh Linh

Mã số sinh viên : N21DCCN139

Sinh viên thực hiện : Nguyễn Tấn Phúc

Mã số sinh viên : N21DCDK022

Lớp : E21CQCNTT01-N

Hệ : Đại học chính quy

TP. HỒ CHÍ MINH, NĂM 2025



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

***Đề tài : “Xây dựng ứng dụng AI nhận
biết thực phẩm và tư vấn dinh dưỡng”***

Người hướng dẫn : Tiến sĩ Nguyễn Hồng Sơn

Sinh viên thực hiện : Phạm Thanh Linh

Mã số sinh viên : N21DCCN139

Sinh viên thực hiện : Nguyễn Tấn Phúc

Mã số sinh viên : N21DCDK022

Lớp : E21CQCNTT01-N

Hệ : Đại học chính quy

TP. HỒ CHÍ MINH, NĂM 2025

Acknowledgement!

My team has chosen the topic " Developing an AI application for food recognition and nutrition advice." to complete our University Graduation Project at the Posts and Telecommunications Institute of Technology, Ho Chi Minh City Campus.

To complete this project, we would like to express our sincere gratitude to our advisor, PhD. Nguyen Hong Son from the Faculty of Information Technology 2, for his dedicated support, guidance, and assistance throughout the process of completing this program and project.

This project aims to develop an Artificial Intelligence application that combines Large Language Models, Retrieval-Augmented Generation (RAG), Computer Vision to support dietary management and nutrition consultation. The system is designed with a user-friendly interface and practical features, enabling users to plan meals efficiently, access reliable nutritional information, and receive personalized dietary recommendations. We would like to acknowledge the guidance of our instructors and the teamwork of all members, whose dedication has been essential for completing this project.

Throughout the project, our team has learned and applied the knowledge acquired from subjects such as Web Programming, Developing Applications for Android Mobile Devices, System Analysis and Design, Database Management, Artificial Intelligence, Natural Language Processing, Machine Learning, Deep Learning, RAG, and Recommendation Systems to complete the project. This process has not only helped us improve our technical skills but also enhanced our teamwork, problem-solving, and time management abilities. More importantly, it has allowed us to apply an Artificial Intelligence program to practical work in the various departments of our internship company.

Once again, our team would like to extend our deepest thanks!

Ho Chi Minh City, October 20, 2025

Our Team

Pham Thanh Linh and Nguyen Tan Phuc

CONTENTS

<i>Acknowledgement!</i>	i
CONTENTS	ii
GLOSSARY OF TERMS.....	vi
INTRODUCTION	1
1) Project Overview: “Developing an AI Application for Food Recognition and Nutrition Advice”	1
2) Justification For The Project	1
3) Project Objectives	2
a. Meal Management:	2
b. Nutritional Consultation:	2
c. Analytics & Reporting.....	3
d. Alerts & Reminders	3
e. AI Agent (RAG-enabled)	3
f. Computer Vision Module.....	3
g. Backend & Integration	3
h. Target Users.....	3
4) How Users Will Use the AI Application System.....	3
5) Team's Contribution Compared To Existing Solutions.	3
TEAM MEMBER TASK ASSIGNMENTS	5
CHAPTER 1: THEORETICAL FOUNDATION	13
1.1 Concept of AI	13
1.2 Concept of Machine Learning	13
1.3 Concept of Deep Learning.....	14
1.4 The Evolution of LLMs.....	15
1.5 Applications Of LLMs in The Project.....	15
1.6 Concepts of AI Chatbot	15
1.6.1 Definition of AI Chatbot	15
1.6.2 Classification of AI Chatbots: Rule-based, Retrieval-based, and Generative AI Chatbots.....	15
1.6.3 Definition of AI Agent:	16
1.7 Why RAG, Not Fine-Tuning A Pre-Trained Model?.....	16
1.8 Concept of Computer Vision.....	17

1.9 Programming Tools and Tech Stacks.....	18
1.9.1 Introduction to Programming Language: Java	18
1.9.2 Introduction to Programming Language: Python.....	19
1.9.3 Introduction to Database Management System.....	19
1.9.4 Introduction to Vector Database.....	20
1.9.5 Concept of developing an Android Backend service with Java Spring Boot.	21
1.9.6 Concept of the Programming Tools: Android Studio	22
1.9.7 Concept of developing an Android front-end.....	22
1.9.8 Introduction to Version Control tools: Git and GitHub	23
1.9.9 Introduction to Langchain.	24
CHAPTER 2: SYSTEM ARCHITECTURE AND DESIGN OF RAG CHATBOT....	25
2.1 AI System Diagram Without RAG, and with RAG:	25
2.2 Overall Architecture of RAG and LLM Integration:	25
2.4 Data Preparation for RAG Chatbot.	26
2.4.1 Data Acquisition.....	26
2.4.2 Semantic Data Engineering.....	27
2.4.3 Chunking Strategy: Row-based Atomic Granularity	28
2.4.4 Vector Embedding and Storage Architecture	29
2.5 Overall Operational Architecture of the RAG Chatbot.....	30
2.5.1 RAG Components and Their Roles.....	31
CHAPTER 3: DEVELOPMENT OF FOOD RECOGNITION AND NUTRIENT ANALYSIS MODULE	33
3.1 Data Preparation	33
3.1.1 Data Mining.....	33
3.1.2 Spatial Labeling with CVAT.....	35
3.1.3 Automated Visual Attribute Extraction (Mixture of Experts)	36
3.1.4 Human-in-the-loop Refinement (Streamlit Tool).....	36
3.1.5 Exploratory Data Analysis (EDA)	38
3.2 Proposed Model Architecture (Faster R-CNN).....	40
CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN	41
4.1 Functional Hierarchy Diagram	41
4.2 Functional Decomposition Diagram.....	41

4.2.1 User Authentication.....	41
4.2.2 Dashboard.....	42
4.2.3 Scanning Food	42
4.2.4 Lucfin Chabot.....	43
4.3 Use Case Diagram	44
4.3.1 Use case for User.....	44
CHAPTER 5: APPLICATIONDEVELOPMENT	45
5.1 SplashScreen and Onboarding Screen.....	45
5.2 Login and Register Screen.....	46
5.3 Dashboard and Add-meal Screen	47
5.4 Scanning food and Scanning food Rresult Screen	48
5.5 Diet detail and Favorite diet screen	49
5.6 Lucfin Chatbot UI: Start Screen and Message Screen	50
5.7 Lucfin Chatbot UI: Gen Answer and History Screen.....	51
CHAPTER 5: FUTURE DEVELOPMENT DIRECTIONS.....	52
CONCLUSION	52
REFERENCES	53

LIST OF FIGURES AND TABLES

Table 1: Glossary of Terms.....	vi
Table 2: Team member task assignments	5
Table 3: Comparison Rof AG and Fine-tuning a Pre-Trained Model.....	16
Table 4: :RAG Components and Their Roles in Explanation	31
Figure 1: Concept of Computer Vision	17
Figure 2: Java Programming Language	18
Figure 3: Python Programming Language	19
Figure 4: MySQL Database Management	19
Figure 5: Vector Database Illustration	20
Figure 6: Java Spring Boot	21
Figure 7: Android Studio	22
Figure 8: Git and Github	23
Figure 9: Langchain	24
Figure 10: AI System Diagram for Departmental Support Without RAG, and with RAG [3]	25
Figure 11: Overall Architecture of RAG and LLM Integration	25
Figure 12: Raw Data	26
Figure 13: Key Data Fields Selected for the RAG System	27
Figure 14: Demonstration of Semantic Data Engineering Process	28
Figure 15: Vector Embedding Process. The system utilizes GPU acceleration to encode processed semantic nodes into the ChromaDB vector store.	29
Figure 16: Overall Operational Architecture of the RAG Advance Chatbot Project [4]	30
Figure 17: Data Preparation and Processing Pipeline	33
Figure 18: Vietnamese Broken Rice Ribs (Suon)	34
Figure 19: Tofu (Dau Hu)	34
Figure 20: Fried Fish Cake (Cha Ca).	34
Figure 21: Spatial Labeling with CVAT	35
Figure 22: Automated Visual Attribute Extraction using Qwen2-VL-7B 16.6GB	36
Figure 23: Run Tool App Streamlit	37
Figure 24: Streamlit Tool for Correction Json File	37
Figure 25 <i>Top Features Summary - Environmental Attribute Distribution</i>	38
Figure 26: <i>Food-Specific Attribute Analysis</i>	39
Figure 27: Faster R-CNN is a single, unified network for object detection. The RPN module serves as the ‘attention’ of this unified network. [11]	40
Figure 28: Functional hierarchy diagram	41
Figure 29: User authentication	41
Figure 30: Dashboard	42
Figure 31: Scanning Food AI	42
Figure 32: Lucfin RAG Chatbot	43
Figure 33: Use-case for User	44
Figure 34: SplashScreen and Onboarding Screen	45
Figure 35: Login Screen	46
Figure 36: Register Screen	46
Figure 37: Dashboard Screen	47
Figure 38: Add-meal Screem	47
Figure 39: Scanning food screen	48
Figure 40: Scanning food Rresult Screen	48
Figure 41: Diet detail Screen	49
Figure 42: Favorite diet screen	49
Figure 43: Start chat Screen	50
Figure 44: Message Screen	50
Figure 45: Gen Answer Screen	51
Figure 46: Chat History Screen	51

GLOSSARY OF TERMS

Table 1: Glossary of Terms

Term	Full meaning	Explanation
AI	Artificial Intelligence	A field of computer science that studies and develops systems capable of performing tasks that require human intelligence.
ANI	Artificial Narrow Intelligence	AI is designed to perform a specific task.
API	Application Programming Interface	A set of definitions and protocols that allows applications to communicate with each other.
CVAT	Computer Vision Annotation Tool	An open-source, web-based tool used to manually annotate images (labeling bounding boxes) for training the object detection model
COCO	Common Objects in Context	A standardized data format for object detection datasets, used to structure the bounding box annotations exported from CVAT.
CRUD	Create, Read, Update, Delete	The four basic operations for managing persistent data in a database (SQL).
CNN	Convolutional Neural Network	A class of deep neural networks commonly used for analyzing visual imagery, serving as the core technology for modern computer vision
ChromaDB	Chroma Database	A vector database is used to store and search embedded data.
IDE	Integrated Development Environment	Software for building applications that combines common developer tools into a single GUI (e.g., Android Studio).
IoU	Intersection over Union	An evaluation metric used to measure the accuracy of an object detector by calculating the overlap area between the predicted box and the ground truth box.
I L VLM	Large Vision-Language Model	Advanced AI models (like Qwen2-VL) capable of processing and understanding both visual (images) and textual information simultaneously.
DL	Deep Learning	A machine learning method that uses multi-layered neural networks to analyze complex data.
JSON	JavaScript Object Notation	A lightweight, easy-to-read/write data format used for data exchange between systems.
LLM / LLMs	Large Language Model(s)	An AI model trained on a massive amount of text data, capable of processing and generating natural language.
mAP	mean Average Precision	A standard performance metric for object detection models (like Faster R-CNN) that measures the average precision across all categories.
ML	Machine Learning	A subfield of AI that allows computers to learn from data without explicit programming.
NLP	Natural Language Processing	A field of AI focused on the interaction between computers and human language.
XML	Extensible Markup Language	A markup language used in Android development to define the structure and layout of the User Interface (UI).
RAG	Retrieval-Augmented Generation	A technique that enhances answer generation by retrieving information from external data sources.
R-CNN	Region-based Convolutional Neural Network	family of deep learning models designed for object detection, which serves as the foundation for the Faster R-CNN architecture used in this project.
RLHF	Reinforcement Learning from Human Feedback	A method for fine-tuning AI models based on human evaluation.
RPN	Region Proposal Network	A component of the Faster R-CNN architecture that scans images to generate "proposals"—regions likely to contain objects—before classification
SOTA	State-of-the-Art	Refers to the highest level of development or the best-performing technology/model available at the current time (e.g., Qwen2-VL).
UI	User Interface	The visual part that allows users to interact with a system.
Vector DB	Vector Database	A database that stores vector embeddings for semantic search.

INTRODUCTION

1) Project Overview: “Developing an AI Application for Food Recognition and Nutrition Advice”

The project titled “Developing an AI Application for Food Recognition and Nutrition Advice” aims to design and implement an intelligent application that helps users manage their daily meals, monitor nutritional intake, and receive personalized dietary recommendations. With the increasing prevalence of sedentary lifestyles, particularly among office workers and IT professionals, there is a growing need for accessible and evidence-based solutions to promote healthier eating habits.

The proposed system integrates multiple Artificial Intelligence techniques to deliver a comprehensive solution. A Recommendation System suggests personalized meal plans tailored to individual health goals, medical conditions, and professional contexts. An AI Agent enhanced with Retrieval-Augmented Generation (RAG) provides accurate and reliable nutritional knowledge, enabling users to query diet-related questions and receive evidence-backed responses. In addition, a Computer Vision module allows users to capture images of their meals and automatically estimate calorie counts and nutrient composition, thereby reducing the burden of manual tracking.

Beyond meal management, the system incorporates features that enhance usability and engagement. Users can generate detailed nutritional reports and analytics, set alerts and reminders when their diets deviate from personal health goals, and interact through social feed by sharing meal posts or exploring posts from similar users. These features aim to balance health-focused guidance with interactive experiences, thereby encouraging long-term adherence to healthier eating practices.

Overall, the project contributes both a practical application for end-users and a research-driven exploration of integrating Large Language Models, RAG, Recommendation Systems, and Computer Vision into a unified dietary management platform. By addressing gaps in personalization, usability, and scientific grounding, the system is expected to offer significant improvements over existing meal planning and nutrition assistance tools.

2) Justification For The Project

Diet and nutrition play a central role in maintaining overall health, preventing chronic diseases, and sustaining productivity. However, modern lifestyles, particularly for office workers and IT professionals, are often associated with prolonged sitting, irregular meal schedules, and limited nutritional awareness. These factors significantly increase the risk of obesity, diabetes, cardiovascular diseases, and kidney-related disorders. Despite the availability of dietary guidelines, individuals frequently struggle to translate abstract recommendations into daily food choices that fit their routines and health conditions.

Existing mobile and web-based nutrition applications provide partial solutions, such as calorie counting, meal logging, or recipe suggestions. However, most systems remain limited in three key aspects: (i) lack of personalization for specific occupations or medical conditions, (ii) high reliance on manual input, which reduces long-term adherence, and (iii) limited scientific grounding in recommendations and responses. Few systems combine advanced Artificial Intelligence methods such as Recommendation

Systems, Retrieval-Augmented Generation (RAG), AI Agents, and Computer Vision into a single platform.

The justification for this project lies in its attempt to close these gaps. By integrating state-of-the-art AI technologies with a user-friendly design, the proposed system can provide evidence-based nutritional advice, automate the tracking of dietary intake, and deliver personalized meal plans tailored to user needs. Beyond individual benefits, the system contributes to public health by addressing dietary challenges in sedentary populations and offering a scalable, intelligent solution that bridges nutritional science with daily life.

3) Current Situation

In recent years, a wide range of mobile and web-based applications for diet tracking and nutrition management have emerged. Popular platforms allow users to record daily meals, calculate calories, and explore recipe databases. Some advanced applications incorporate barcode scanning or partial image recognition to simplify data entry. While these tools have attracted large user bases, their functionalities are typically fragmented and not deeply personalized to specific lifestyles or health conditions.

From a research perspective, Artificial Intelligence has been increasingly applied to the food and nutrition domain. Computer vision models have been trained to recognize dishes and estimate portion sizes, though their accuracy often declines in uncontrolled environments or diverse culinary contexts. Recommender systems are widely used in e-commerce and entertainment, yet their adoption in nutrition remains limited to basic “similar recipe” suggestions. More recently, the rise of Large Language Models and RAG techniques has enabled conversational systems in healthcare and education, but few practical implementations exist in the domain of dietary consultation.

Despite these advancements, there remains a clear gap in integrated systems that combine meal management, personalized nutrition advice, and user engagement features into a single application. Current solutions often require extensive manual input, provide generic recommendations, or lack transparency in how suggestions are generated. As a result, users face difficulties in maintaining long-term adherence to healthier diets, and the potential of AI in this domain remains underutilized.

3) Project Objectives

The project aims to create an AI-driven system that assists individuals in managing daily meals and achieving balanced nutrition. The objectives are as follows:

a. Meal Management:

- Allow users to log daily meals with flexible input methods (text, photo).
- Provide timeline views and visual progress charts to help track eating patterns.
- Support exportable meal histories for statistical review and long-term monitoring.

b. Nutritional Consultation:

- Offer personalized meal suggestions adapted to user goals and medical conditions (e.g., weight loss, diabetes, cardiovascular, and kidney-related concerns).
- Display ingredient-level nutritional facts to increase transparency and informed choices.
- Provide dynamic dietary plans tailored to office workers and professionals with sedentary lifestyles.

c. Analytics & Reporting

Generate aggregated reports on calorie intake, macronutrient balance, and selected micronutrients.

Allow flexible reporting over custom intervals (daily, weekly, monthly).

Visualize deviations from nutritional goals to aid corrective action.

d. Alerts & Reminders

Send notifications for skipped meals, unhealthy trends, or goal deviations.

Enable scheduled reminders aligned with individual routines and working hours.

e. AI Agent (RAG-enabled)

Deploy a chatbot capable of answering nutrition-related questions based on evidence and curated references.

Focus on practical guidance for office workers with limited physical activity.

Continuously update the knowledge base through retrieval-augmented generation.

f. Computer Vision Module

Recognize dishes and estimate calories and nutrients directly from images.

Optimize models for on-device or edge processing to ensure speed and privacy.

g. Backend & Integration

Maintain a vector database for embeddings and nutrition knowledge retrieval. Ensure compliance with data privacy and authentication standards.

h. Target Users

General users seeking convenient, evidence-based dietary support and interested in structured meal planning and community interaction.

4) How Users Will Use the AI Application System.

Users begin by creating a personal profile, where they can enter basic information such as age, weight, dietary goals, and any health-related conditions. From there, the application offers several ways to interact with the system.

When eating, users can simply log their meals by either selecting from a food database or uploading a photo of their dish. The system will provide an overview of calories and key nutrients, making it easy for users to monitor their intake. If they are unsure what to eat, they can request personalized meal suggestions aligned with their health goals, preferences, and lifestyle.

Users can also ask the application-specific questions about nutrition or food choices, receiving clear, evidence-based responses. For those who want ongoing guidance, the app generates daily or weekly summaries of nutritional progress, highlights unhealthy patterns, and sends reminders to stay on track.

Beyond individual use, the application includes a social space where users can share meals, exchange ideas, and follow others for inspiration. This combination of personal tracking, smart guidance, and community support makes the system both practical and engaging for everyday use.

5) Team's Contribution Compared To Existing Solutions.

Unlike existing calorie-tracking or generic diet applications, our project contributes a comprehensive AI-driven system that integrates recommendation, retrieval-augmented reasoning, computer vision, and social interaction into a single modular architecture.

The first distinctive contribution is an occupation and condition-aware recommendation engine. Instead of relying solely on collaborative or content-based filtering, our system incorporates occupation-specific lifestyle patterns (e.g., IT professionals with long sedentary hours), health-related constraints (e.g., diabetes, cardiovascular disease, kidney disease), and personal food preferences. This hybrid design enables

recommendations that are not only personalized but also medically safe, thereby reducing rejection rates of suggested plans.

The second major contribution lies in the RAG-enabled AI agent with provenance support. In contrast to common chatbots that generate responses without verifiable sources, our agent retrieves knowledge from a curated nutrition corpus and presents answers with citations, confidence scores, and temporal validity. This mechanism mitigates hallucinations, enhances transparency, and builds user trust, particularly when health-related guidance is involved.

Third, our system advances computer vision for dietary assessment by moving beyond simple food classification. We propose a multi-stage pipeline that performs food segmentation, portion size estimation, and nutrient mapping to yield calorie and nutrient estimates. This design supports both lightweight on-device inference for real-time use and more accurate server-side models for batch analysis. Importantly, we adopt a user-in-the-loop correction approach, allowing active learning and continuous improvement of recognition accuracy, particularly for local cuisines underrepresented in global datasets.

Another contribution is the integration of RAG, recommender systems, and computer vision in a modular yet interconnected architecture. Each module operates independently but also feeds into others: the vision module provides dietary logs to the recommender, while the RAG agent introduces domain knowledge and safety constraints into meal plan generation. The architecture is provenance-aware, auditable, and privacy-preserving, with edge processing and encryption ensuring sensitive health data remains under user control.

We further emphasize explainability and transparency in AI outputs. Each recommendation is accompanied by a rationale derived from feature attribution, while chatbot answers display retrieved sources and confidence levels. This explainability ensures that users not only receive personalized guidance but also understand why it is relevant and how reliable it is.

Our system also introduces a localized dataset construction protocol, addressing the frequent mismatch between international food datasets and regional cuisines. By developing annotated datasets of local dishes with portion and nutrient information, the system achieves higher accuracy for target users, especially office workers in Vietnam and Southeast Asia.

Finally, we contribute to user engagement and safety through a social feed combined with health-aware filtering. Unlike conventional health apps that merely provide community features, our system ensures that shared meal posts are moderated to prevent harmful or misleading dietary content for users with specific medical conditions. Evaluation of the system will not only cover model accuracy but also user-centered metrics such as adherence to dietary recommendations, reduction in calorie estimation error, and perceived trust in AI suggestions.

In summary, our team's contribution extends beyond developing another nutrition-tracking app. By combining occupation-aware recommendations, evidence-grounded AI dialogue, advanced food recognition, transparent system design, and health-aware social interaction, we deliver a platform that both advances research in applied AI for nutrition and addresses practical gaps in current solutions.

TEAM MEMBER TASK ASSIGNMENTS

Table 2: Team member task assignments

No.	Main Task	Detailed Content	Responsible Member
1	Literature Review	Study the fundamentals of the AI Agent System. Research these similarity systems that exist for reference. Research and survey the practical needs for the application.	Whole Team
2	Design UX/UI	Design UX/UI for the Android Application	Whole Team
3	Develop Frontend	Develop a Application Front-end in Android Studio	Nguyễn Tấn Phúc
4	Develop Backend	Developing a Basic Back-end service	Phạm Thanh Linh
5	RAG Chatbot	Develop a RAG chatbot into an Android application for answering nutrient knowledge	Nguyễn Tấn Phúc
6	Model Computer Vision	Develop a Computer Vision model to detect and evaluate the food nutrients	Phạm Thanh Linh
7	Preparation data for RAG Chatbot	Food dataset for chunking, augmentation LLMs	Nguyễn Tấn Phúc
8	Preparation data for AI Camera Food Recognition	Preparation dataset for training Computer Vision model	Whole Team
9	Evaluation RAG Chatbot	Evaluation RAG system by the metrics	Nguyễn Tấn Phúc
10	Evaluation Computer Vision Model	Evaluation result of YOLO model	Phạm Thanh Linh

CHAPTER 1: THEORETICAL FOUNDATION

1.1 Concept of AI

Artificial Intelligence (AI) is a broad and fundamental field of computer science that focuses on studying and building computer systems capable of performing tasks that typically require human intelligence. According to John McCarthy, one of the co-founders of the term in 1956, AI is "the science and engineering of making intelligent machines."

The capabilities that AI aims to emulate include learning (acquiring information and rules for using that information), reasoning (using rules to reach approximate or definite conclusions), self-correction, perception (using artificial senses like cameras and microphones), and natural language processing.

Basically, AI can be classified based on its capabilities into three main types:

- **Artificial Narrow Intelligence (ANI):** This is the only type of AI that humans have achieved to date. Narrow AI is designed and trained to perform a specific task or a closely related set of tasks. It can outperform humans in its specialized domain but is unable to apply its knowledge outside of its programmed scope. Examples include facial recognition software, virtual assistants (Siri, Alexa), product recommendation systems on e-commerce sites, and the chatbot system developed in this project.
- **Artificial General Intelligence - AGI:** This is the next level of AI, where a machine possesses intelligence equivalent to that of a human. An AGI system can understand, learn, and apply its knowledge to solve any problem that a human can. AGI remains a research goal and has not yet been realized.
- **Artificial Superintelligence - ASI:** This is a hypothetical form of AI that surpasses the intelligence of the most brilliant human minds in every field, from scientific creativity to social intelligence.

Within the scope of this project, we focus on applying **Artificial Narrow Intelligence**, specifically techniques in the fields of **Machine Learning** and **Deep Learning**, to solve a practical problem within an enterprise.

1.2 Concept of Machine Learning

For a computer system to "understand" and "communicate" in natural language within a specialized context like employee support, it cannot rely on fixed, hard-coded rules. Instead, the system must have the ability to learn from the text data itself. **Machine Learning (ML)** is the subfield of AI that provides the methods and tools to perform this learning process.

At its core, Machine Learning enables computers to automatically recognize patterns and rules from data to make decisions or predictions, without explicit human programming. In the field of **Natural Language Processing (NLP)**, this is the key to solving complex problems.

The main Machine Learning methods, as applied in the context of this project's text processing, can be understood as follows:

- **Supervised Learning:** This is a foundational method for many language tasks. A model learns from a dataset of text that has been manually labeled by humans.

- **Relevant Application to the Project:** Classifying documents by topic (e.g., contract, report, or procedure), machine translation, or recognizing important entities within a sentence (project names, dates, terms).
- **Unsupervised Learning:** This method is particularly useful when working with an enterprise's massive, unlabeled document repository. The model autonomously discovers the underlying structure of the data.
- **Relevant Application to the Project:** Automatically clustering documents with similar topics (topic modeling), or more importantly, learning **word embeddings**—the foundation for a computer to understand the semantic meaning of words and sentences.
- **Reinforcement Learning:** Although less common in traditional NLP tasks, this method is becoming central to fine-tuning modern chatbots. The model learns through a process of trial and error, receiving a "reward" for good answers and a "penalty" for poor ones.

Relevant Application to the Project: Advanced techniques like **Reinforcement Learning from Human Feedback (RLHF)** are used to make chatbots (like ChatGPT) more helpful, truthful, and safe.

The development of these Machine Learning methods, especially supervised learning on massive text datasets, laid the groundwork for the emergence of **Deep Learning** and subsequently **Large Language Models (LLMs)** the core technology applied in this project to create a truly intelligent chatbot.

1.3 Concept of Deep Learning

Deep Learning (DL) is a specialized subfield of Machine Learning that uses complex *Artificial Neural Networks* with multiple layers to analyze data. The term "deep" refers to the large number of these layers. While traditional Machine Learning models can be limited in their ability to handle the complexity of raw data (such as text or images), Deep Learning excels by automatically learning various levels of abstract representations.

For the natural language processing task in this project, the power of Deep Learning is demonstrated by:

- The model's ability to automatically learn from individual characters and words in the initial layers.
- In deeper layers, it combines this knowledge to recognize phrases and grammatical structures.
- In the deepest layers, it can grasp the semantic meaning, nuance, and complex relationships within an entire passage of text.

One of the most significant and revolutionary advantages of Deep Learning over traditional Machine Learning is automatic feature extraction. Instead of engineers painstakingly designing features for text data (like word frequency counts, n-grams), Deep Learning models can learn the most important features directly from the input data itself, optimizing performance and minimizing human intervention.

The true breakthrough in applying Deep Learning to NLP came from an architecture called the Transformer, introduced in 2017. The core of the Transformer is the attention mechanism, which allows the model, when processing a word, to "look back" and weigh the importance of all other words in the sentence. This helps it effectively capture context and long-range dependencies, a weakness of previous architectures.

The Transformer architecture is the foundation for almost all modern Large Language Models (LLMs). It is thanks to the power of Deep Learning and Transformer architecture that researchers have been able to build colossal language models, creating a revolution in how computers interact with humans. We will explore LLMs in more detail in the next section.

1.4 The Evolution of LLMs

Transformer architecture is the foundation for almost all modern **Large Language Models (LLMs)** [1]. It is thanks to the power of Deep Learning and Transformer architecture that researchers have been able to build colossal language models, creating a revolution in how computers interact with humans. We will explore LLMs in more detail in the next section.

1.5 Applications Of LLMs in The Project.

A Large Language Model (LLM) is a Deep Learning model, typically based on the Transformer architecture, trained on a massive corpus of text data. The most basic objective of its training is to learn how to predict the next word in a sequence of text. For example, given the input "It's a beautiful day, I want to go...", the model learns to predict likely next words such as "for a walk," "fishing," "to the beach," etc.

Although this task may sound simple, when performed on an immense scale, it gives the model extraordinary characteristics. A model is considered "Large" when it possesses the following attributes:

Massive Number of Parameters: Modern LLMs have billions to trillions of parameters. These parameters can be seen as the model's brain, where all the knowledge, syntactic patterns, and semantic understanding it has learned from the data are stored.

Immense Training Data: They are trained on vast datasets of text, including a significant portion of the internet's content, such as books, news articles, Wikipedia, and source code. This provides them with a profound knowledge base across almost every domain.

Emergent Abilities: This is the most remarkable and crucial characteristic of LLMs. Due to their immense scale, they spontaneously develop new capabilities for which they were not directly trained. These emergent abilities include text summarization, translation, question-answering, creative writing, programming, and even logical reasoning. It is these emergent abilities that have transformed LLMs from simple word prediction models into versatile artificial brains.

1.6 Concepts of AI Chatbot

1.6.1 Definition of AI Chatbot

Definition: An AI Chatbot is a computer program designed to simulate human conversation through text or voice. The main goal of a chatbot is to "converse" and "provide information" based on a specific knowledge source. It acts as a communication interface, helping users interact with a computer system in a natural way.

1.6.2 Classification of AI Chatbots: Rule-based, Retrieval-based, and Generative AI Chatbots.

Rule-based Chatbots:

How they work: They operate based on a pre-programmed script and logical flow (if-then). They can only answer questions that fall within that script.

Advantages: Easy and inexpensive to develop; answers are consistent and predictable.

Disadvantages: Very rigid; cannot handle out-of-scope questions; lacks context understanding; provides a less natural user experience.

Retrieval-based Chatbots:

How they work: They use information retrieval techniques (like keyword or semantic search) to find the most suitable answer from a database of pre-existing Q&A pairs or documents. The RAG system in this project is a highly advanced form of this type of chatbot.

Advantages: Answers are based on real, reliable data sources and do not fabricate information.

Disadvantages: Cannot generate completely new answers; quality depends on the availability of source data.

Generative Chatbots:

How they work: They use Large Language Models (LLMs) to "generate" new answers based on their vast knowledge learned. ChatGPT is the most typical example.

Advantages: Extremely flexible; can converse on any topic; generates natural and human-like responses.

Disadvantages: Poses a risk of "hallucinations", fabricating non-existent information.

Project Relevance: The RAG Chatbot system intelligently combines the Retrieval-based approach (using RAG for accuracy) and the Generative approach (using an LLM to create natural, coherent answers).

1.6.3 Definition of AI Agent:

AI agents [5] are software systems that use AI to pursue goals and complete tasks on behalf of users. They show reasoning, planning, and memory and have a level of autonomy to make decisions, learn, and adapt.

Their capabilities are made possible in large part by the multimodal capacity of generative AI and AI foundation models. AI agents can process multimodal information like text, voice, video, audio, code, and more simultaneously; can converse, reason, learn, and make decisions. They can learn over time and facilitate transactions and business processes. Agents can work with other agents to coordinate and perform more complex workflows.

1.7 Why RAG, Not Fine-Tuning A Pre-Trained Model?

When aiming for a **Large Language Model (LLM)** to "understand" an enterprise's internal data, there are two primary approaches: **Fine-tuning** and **Retrieval-Augmented Generation (RAG)**. Within the scope of this project, our team chose RAG as the core architecture for the following strategic reasons:

Table 3: Comparison of AG and Fine-tuning a Pre-Trained Model

Criterion	Retrieval-Augmented Generation (RAG)	Fine-tuning	Project's Choice
Knowledge Update	Very easy and fast. Only requires updating documents in the database.	Difficult and costly. Requires retraining the entire model when new data is available.	RAG is suitable for a corporate environment where data changes continuously.

Hallucination Prevention	Highly effective. Answers are constrained by the retrieved information, and sources can be cited for verification.	Higher risk. The model can "misremember" or fabricate information if the fine-tuning data is incomplete.	RAG ensures accuracy and reliability, which are crucial factors in an enterprise.
Cost & Resources	Lower. The primary costs are for API usage and vector storage; it does not require powerful GPUs for training.	Very high. Requires powerful GPU infrastructure and significant time and cost for each training session.	RAG is a feasible and economical choice for a graduation project and for small-to-medium-sized enterprises.
Transparency & Traceability	Very good. It can pinpoint which specific text segment or document was used to generate the answer.	Nearly impossible. Operates as a "black box," making it hard to understand why the model provided a certain answer.	RAG meets the requirement for transparency when information needs to be cross-checked.
Style Modification	Ineffective. RAG primarily changes the model's "knowledge" rather than its "behavior" or "style."	Effective. Can teach the model to speak in a specific tone, style, or format.	The project prioritizes knowledge accuracy over style modification.

1.8 Concept of Computer Vision

Computer Vision is a field of artificial intelligence (AI) that trains computers to interpret and understand the visual world. Using digital images from cameras, videos, and deep learning models, machines can accurately identify and classify objects and then react to what they "see." The ultimate goal of computer vision is to enable machines to replicate the powerful capabilities of human vision, a task that involves not just seeing, but also processing and comprehending visual information.

At its core, computer vision works by applying machine learning algorithms, particularly deep learning models, to visual data. Modern computer vision relies heavily on Convolutional Neural Networks (CNNs), a type of neural network architecture that is exceptionally effective at processing grid-like data such as images.

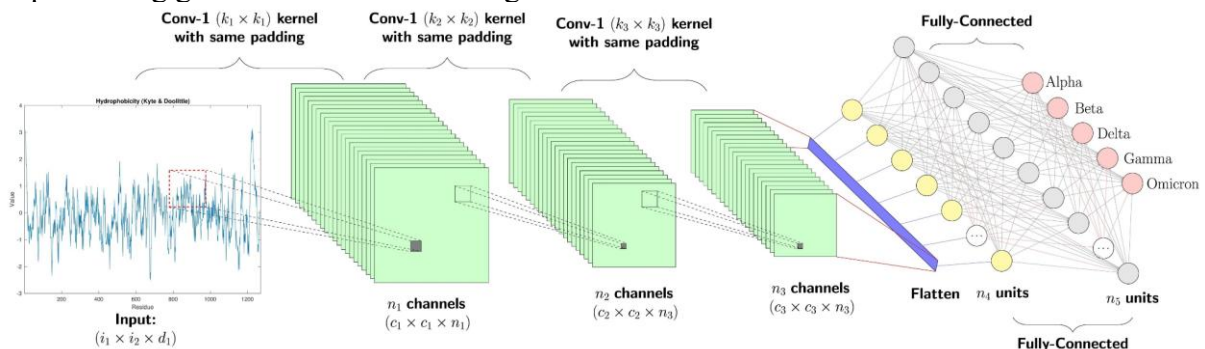


Figure 1: Concept of Computer Vision

The process typically involves several stages: an image is first acquired, then pre-processed to enhance its quality, and finally fed into a trained model which extracts features and makes predictions. This model learns to recognize patterns, textures, shapes, and colors from a

massive dataset of labeled images, much like how a human learns to identify objects through experience.

Computer vision encompasses several key tasks that enable complex applications. The most fundamental tasks include:

- **Image Classification:** This involves assigning a single label to an entire image. For example, an image might be classified as "a plate of spaghetti" or "a bowl of pho."
- **Object Detection:** A more advanced task, object detection identifies the presence and location of multiple objects within an image. It draws a bounding box around each distinct object and assigns a label to it, such as identifying a "chicken breast," "broccoli," and "rice" on a single plate.
- **Image Segmentation:** This task provides the most detailed understanding by classifying each individual pixel of an image. It essentially creates a pixel-level mask for each object, allowing for precise shape recognition and analysis, which is crucial for estimating food portion sizes.

In the context of this project, computer vision is the foundational technology for the food recognition module. Users will upload a photo of their meal, and a computer vision model will analyze it to identify all the food items present. By leveraging tasks like object detection and segmentation, the application can accurately determine what the user is eating. This visual data is then converted into structured information, which serves as the input for the subsequent stages of nutritional analysis and dietary advice.

1.9 Programming Tools and Tech Stacks

1.9.1 Introduction to Programming Language: Java



Figure 2: Java Programming Language

Java is a high-level, class-based, object-oriented programming language designed to have as few implementation dependencies as possible. It is a general-purpose language intended to let application developers "write once, run anywhere" (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation. This platform independence is achieved through the Java Virtual Machine (JVM), which acts as an interpreter between the Java code and the underlying operating system. Key features of Java include its strong memory management through an automatic garbage collector, a robust security model, and a rich standard library (API). Due to its reliability and scalability, Java is widely used for developing enterprise-level backend systems, Android mobile applications, and large-scale web applications.

1.9.2 Introduction to Programming Language: Python



Figure 3: Python Programming Language

Python is a high-level, interpreted, general-purpose programming language renowned for its design philosophy that emphasizes code readability and simplicity. Unlike compiled languages like Java, Python code is executed directly by an interpreter, which simplifies the development and testing process. Its syntax is clean and elegant, allowing developers to express concepts in fewer lines of code than would be possible in many other languages.

Key characteristics of Python include its dynamic typing, where variable types are determined at runtime, offering greater flexibility. It also features automatic memory management and a massive standard library, often referred to as its "batteries-included" philosophy, which provides tools for a wide range of tasks. Due to its powerful libraries (such as TensorFlow, PyTorch, and Scikit-learn) and ease of use, Python has become the dominant language for artificial intelligence, machine learning, data science, and scientific computing. It is the primary language chosen for the implementation of this project because of its robust ecosystem for developing AI-driven applications.

1.9.3 Introduction to Database Management System.



Figure 4: MySQL Database Management

A Database Management System (DBMS) is a software system that enables users to create, maintain, and control access to a database. It acts as an intermediary between the user and the database, ensuring that data is consistently organized and remains easily accessible. The most common type of DBMS is the Relational Database Management System (RDBMS), which stores data in a structured format using tables with rows and columns.

SQL (Structured Query Language) is the standard programming language used to communicate with and manage data within an RDBMS. It is not a DBMS itself but rather the language used to query, manipulate, and define the data. SQL provides a standardized syntax for performing essential operations, known as CRUD (Create, Read, Update, Delete). Key commands include SELECT to retrieve data, INSERT to add new records, UPDATE to modify existing records, and DELETE to remove data. For this project, an SQL-based

database is essential for persistently storing critical information, such as user profiles, dietary goals, meal logs, and the comprehensive nutritional details of various food items.

1.9.4 Introduction to Vector Database.

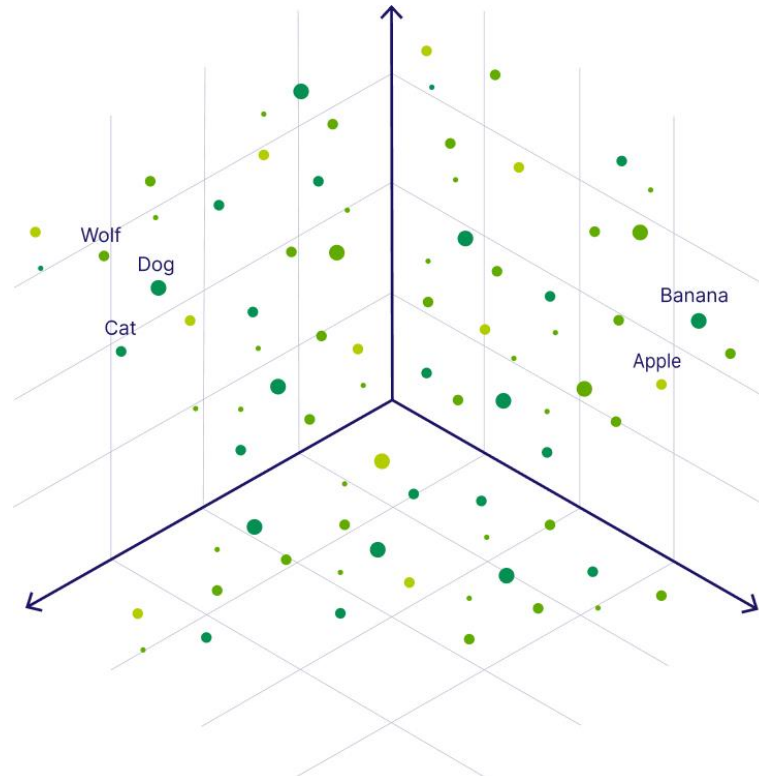


Figure 5: Vector Database Illustration

A Vector Database is a specialized type of database designed to store, manage, and query high-dimensional data points called vector embeddings. Unlike traditional relational databases (like SQL) that are optimized for structured data and exact-match queries, vector databases excel at performing incredibly fast similarity searches. The core purpose is not to find data that matches a specific keyword, but to find data that is "semantically similar" or "contextually related."

The foundation of a vector database lies in the concept of embeddings. Unstructured data—such as text from a nutritional guide, an image of a meal, or a user's queries converted by a machine learning model into a numerical vector. This vector captures the essential meaning or features of the original data. For example, the vectors for "high-protein snacks" and "post-workout meals" would be located close to each other in the vector space because they are conceptually related.

When a query is made, it is also converted into a vector. The database then uses highly efficient algorithms, such as Approximate Nearest Neighbor (ANN) search, to find the vectors in its index that are closest to the query vector. The result is a ranked list of the most similar items.

Vector databases are fundamental to modern AI applications, particularly Retrieval Augmented Generation (RAG). In a RAG system, the vector database acts as the long-term memory for a Large Language Model (LLM). It stores a vast knowledge base (e.g., nutritional information, recipes, dietary articles) as embeddings. When a user asks a question, the system first queries the vector database to retrieve the most relevant documents. These documents are then passed to the LLM as context, enabling it to generate an accurate, fact-based, and informed answer. For this project, a vector database is crucial for building the intelligent agent that provides reliable nutritional advice.

1.9.5 Concept of developing an Android Backend service with Java Spring Boot.



Figure 6: Java Spring Boot

In modern mobile application architecture, the Android application itself serves as the client-side or frontend. It is responsible for the user interface (UI) and user experience (UX). However, the core business logic, data processing, and persistent storage are handled by a server-side component known as the backend. For this project, the backend will be developed using Java Spring Boot.

Java Spring Boot is a powerful, open-source framework built on top of the popular Spring framework. Its primary goal is to simplify the creation of stand-alone, production-grade, Spring-based applications that you can "just run." It achieves this through its "convention over configuration" philosophy, which means it makes intelligent assumptions about what a developer needs, drastically reducing boilerplate code and manual configuration.

The communication between the Android client and the Spring Boot backend is established through a REST (Representational State Transfer) API. This architecture works as follows:

- **HTTP Request:** The Android application sends an HTTP request to a specific URL endpoint on the backend server. The request can be to retrieve data (GET), create new data (POST), update existing data (PUT), or delete data (DELETE).
- **Backend Processing:** The Spring Boot application receives this request, processes it, performs the necessary business logic (e.g., querying the database, calculating nutritional values), and prepares a response.
- **HTTP Response:** The backend sends an HTTP response back to the Android app, typically containing the requested data formatted in JSON (JavaScript Object Notation), which is a lightweight, human-readable format that is easy for mobile applications to parse and use.

The key responsibilities of the Spring Boot backend in this project include:

- **User Authentication and Management:** Securely handling user registration, login, and profile management using modules like Spring Security.
- **Database Abstraction:** Interacting with both the SQL database (for user data, meal logs) and the Vector Database (for the RAG system) to store and retrieve information.
- **Business Logic Execution:** Housing all the complex server-side logic, such as processing the output from the computer vision model, aggregating nutritional data, and powering the recommendation engine.
- **Providing a Secure API:** Exposing a well-defined set of API endpoints for the Android application to consume, ensuring data is transferred securely and efficiently.

By using Spring Boot, we can build a backend that is robust, scalable, and easy to maintain, allowing the Android app to remain focused on its primary role of delivering an excellent user interface.

1.9.6 Concept of the Programming Tools: Android Studio



Figure 7: Android Studio

Android Studio is the official Integrated Development Environment (IDE) for Google's Android operating system, designed specifically for Android app development. Built on JetBrains' IntelliJ IDEA software and endorsed by Google, it provides a unified environment where developers can write, design, test, and debug their applications.

It is a comprehensive toolset that streamlines the entire development lifecycle. At its core, Android Studio features an intelligent code editor that supports Kotlin and Java, offering advanced features like code completion, refactoring, and static code analysis to help developers write high-quality code faster.

Key features that make Android Studio indispensable for this project include:

- **Visual Layout Editor:** This tool allows developers to build user interfaces (UIs) by dragging and dropping components onto a virtual screen. It provides a real-time preview of the app's layout on different screen sizes and automatically generates the corresponding XML code.
- **Fast and Feature-Rich Emulator:** It includes an Android Emulator that can simulate various Android devices, from phones to tablets and wearables. This allows for testing the application on different hardware configurations and Android versions without needing a physical device for each one.
- **Flexible Build System (Gradle):** Android Studio uses Gradle as its build system, which automates the process of compiling code, managing external libraries (dependencies), and packaging the application into an APK (Android Package Kit) for distribution.
- **Real-time Profilers and Debugger:** It comes with a suite of tools for monitoring the app's performance in real-time, including CPU, memory, and network usage. The integrated debugger is essential for identifying and fixing bugs in the code.

For this project, Android Studio will be the primary tool for developing the entire client-side mobile application. It will be used to design the user interface, implement the application logic for capturing images, display nutritional information, and manage all communication with the Java Spring Boot backend via the REST API.

1.9.7 Concept of developing an Android front-end.

The front-end of an Android application is the user interface (UI) the visual layer that users directly see and interact with. For this project, the UI is developed using the traditional and robust approach of defining layouts in XML (eXtensible Markup Language).

XML is a declarative markup language that defines the structure, dimensions, and appearance of all the UI elements on the screen. Each layout file consists of a hierarchy of Views and ViewGroups:

- A View is a fundamental UI component that draws something on the screen the user can interact with. Common examples include Button, TextView (for displaying text), ImageView (for displaying images), and EditText (for user input).
- A ViewGroup is an invisible container that defines the layout structure for other Views and ViewGroups. It organizes the components held within it. Popular ViewGroups include LinearLayout for arranging elements in a single row or column, RelativeLayout for positioning child views relative to each other, and ConstraintLayout, a powerful and flexible layout manager that allows developers to create complex UIs with a flat view hierarchy.

These XML layout files are completely decoupled from the application's logic, which is written in Kotlin or Java. During runtime, the system "inflates" these layout files, converting the XML tags into their corresponding View objects in memory. To interact with these UI elements—for instance, to set text on a TextView or handle a Button click—the developer must create a reference to them in the code. This is traditionally done using the `findViewById()` method or, more modernly and safely, through View Binding, which generates a binding class for each XML file to provide direct, type-safe access to the views.

This XML-based approach is a mature and well-established method for building Android UIs, providing precise control over the layout and maintaining a clear separation between the application's visual presentation and its underlying business logic.

1.9.8 Introduction to Version Control tools: Git and GitHub



Figure 8: Git and Github

Version Control is a system that records changes to a file or set of files over time so that you can recall specific versions later. For software development, this is crucial for tracking code modifications, collaborating with team members, and maintaining a stable project history. Git and GitHub are two foundational tools in modern version control.

Git is a free, open-source distributed version control system (DVCS). Unlike centralized systems, every developer's working copy of the code is also a complete repository with the full history of all changes. This allows for powerful local operations, such as creating branches to work on new features in isolation without affecting the main codebase. Developers can commit changes locally, view project history, and merge branches on their own machines. Git is renowned for its speed, data integrity, and support for non-linear, distributed workflows.

GitHub, on the other hand, is a cloud-based hosting service for Git repositories. It acts as a central remote repository where team members can push their local changes and pull updates from others. GitHub builds on top of Git's core functionality by providing a web-based graphical interface and powerful collaboration features. Key functionalities include:

- Pull Requests: A mechanism for proposing changes and initiating code reviews before merging them into the main branch.
- Issue Tracking: A system for managing tasks, bugs, and feature requests.
- Actions: An automation tool for building, testing, and deploying code directly from the repository.

In essence, Git is the tool for managing code history, while GitHub is the platform for hosting that history and facilitating teamwork. For this project, using Git and GitHub is essential for managing the source code, enabling collaboration between developers, and maintaining a secure, versioned backup of the entire application.

1.9.9 Introduction to Langchain.



Figure 9: Langchain

In the Lucfin project, LangChain serves as the Orchestration Layer, bridging the gap between the raw language model (LLM) and the application logic. While the underlying vector database handles data storage, LangChain manages the dynamic flow of information, ensuring that the interaction between the user and the AI remains coherent, stateful, and logic-driven.

Specifically, LangChain is utilized to manage:

- Interaction Flow: Coordinating the sequence of operations from input processing to response generation.
- Memory Management: Handling short-term conversational history to support multi-turn dialogues.
- Prompt Engineering: Dynamically constructing prompts using templates to guide the LLM's behavior (e.g., switching between "Nutrition Expert" and "Chitchat" personas)

1.0.9.1 Advanced Retrieval Strategy: History-Aware Retriever

A critical challenge in conversational RAG systems is the "contextual ambiguity" in follow-up questions. For example, if a user asks about "Black Chicken Soup" and subsequently asks "Is it fattening?", a standard retriever would fail to identify the subject "it."

To resolve this, we implemented a History-Aware Retriever mechanism using LangChain. The process operates as follows:

1. Contextualization: When a new query is received, the system analyzes it alongside the Chat History.
2. Query Reformulation: An intermediary LLM call rewrites the user's raw query into a Standalone Question.
 - Input: "Is it fattening?" + [History: User asked about Black Chicken Soup]
 - Reformulated Output: "Is eating Black Chicken Herbal Soup fattening?"
3. Precision Retrieval: This standalone question is then used to query the Vector Database, ensuring high retrieval accuracy even when the user uses pronouns or implicit references.

1.9.9.2 Hybrid Integration Strategy (LangChain + LlamaIndex)

To maximize performance, the project adopts a Hybrid Architecture:

- Data Layer (LlamaIndex): Used for efficient data ingestion, node parsing, and vector indexing. LlamaIndex excels at handling structured (CSV) and unstructured data with optimized chunking strategies.
- Application Layer (LangChain): Used for controlling the conversational flow.

- The Bridge: We implemented a custom Retriever Wrapper that converts LlamaIndex's retrieval results into LangChain Document objects. Crucially, this wrapper preserves the Metadata (specifically image links), allowing the application to display visual content alongside textual advice.

CHAPTER 2: SYSTEM ARCHITECTURE AND DESIGN OF RAG CHATBOT

2.1 AI System Diagram Without RAG, and with RAG:

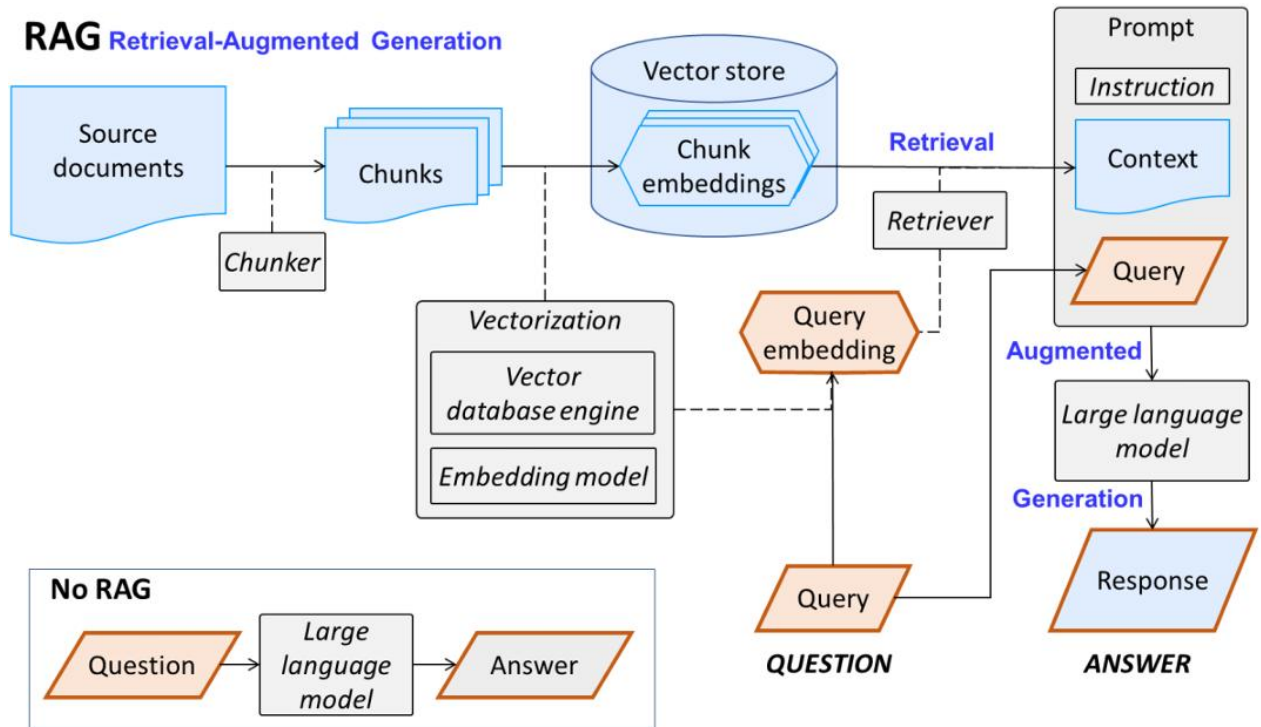


Figure 10: AI System Diagram for Departmental Support Without RAG, and with RAG [3]

2.2 Overall Architecture of RAG and LLM Integration:

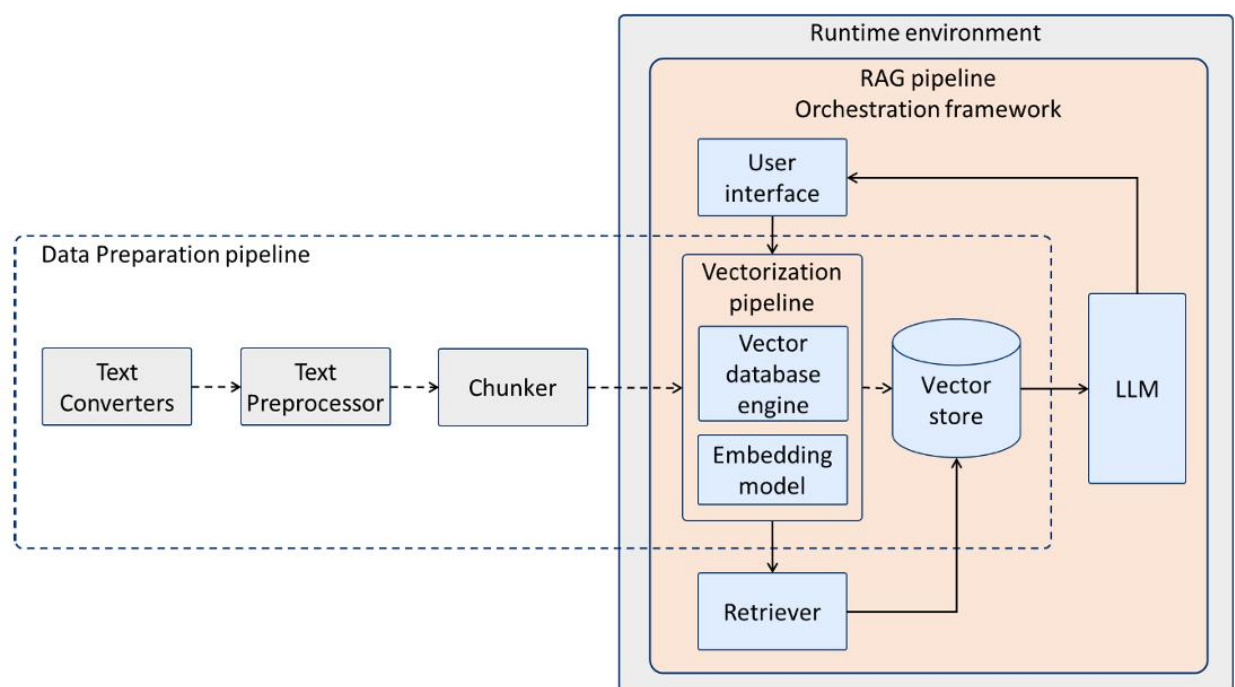


Figure 11: Overall Architecture of RAG and LLM Integration

2.4 Data Preparation for RAG Chatbot.

2.4.1 Data Acquisition

a. Data Source

This research leverages and extends the foundational dataset from the open-source project *"A New Dataset and Empirical Evaluation for Vietnamese Food Recommendation System"* [7]. This was selected as the primary data source because it represents one of the few structured, high-reliability datasets specifically curated for Vietnamese cuisine. While originally designed for Recommender Systems (matrix factorization tasks), its rich attribute set makes it a suitable candidate for adaptation into a Knowledge Base for RAG.

b. Data Profiling

The raw data is acquired in CSV format, where each record represents a unique culinary entity. The dataset is characterized by multi-dimensional attributes, ranging from unstructured textual descriptions to quantitative nutritional metrics.

Based on the raw data sample, the schema is categorized as follows:

- **Identity & Classification:** food_id, dish_name (Entity Name), dish_type (e.g., Savory, Vegetarian), dish_tags.
- **Unstructured Text (Process & Content):** description (Flavor profile), ingredients (Detailed components), cooking_method (Step-by-step guide).
- **Quantitative Metrics (Nutritional Facts):** calories, protein, fat, fiber, sugar, cooking_time, and serving_size.
- **Multimedia Resources:** image_link (URL for visual representation).

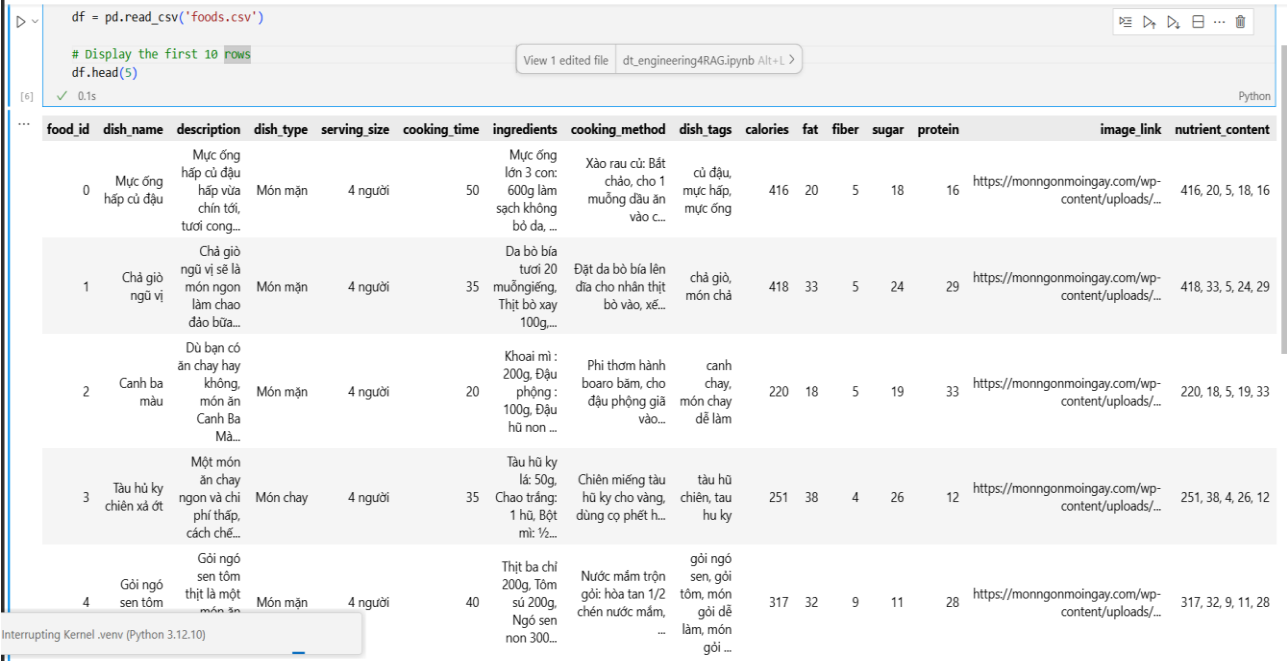


Figure 12: Raw Data

c. Gap Analysis

While the original dataset is robust, we identified specific **Architectural Gaps** when applying it to a RAG framework:

- **Semantic Fragmentation:** The data is structured in a tabular format (row-column) optimized for relational queries. However, RAG relies on semantic vector search. Treating columns like ingredients and cooking_method as isolated vectors would sever the semantic relationship between *what* the dish is made of and *how* it is cooked.
- **Information Noise:** Certain fields, such as internal system IDs (food_id) or redundant tags, do not contribute to the semantic understanding of the Large Language Model (LLM) and would introduce noise into the vector space.

d. Curation Process

To bridge these gaps, a data curation pipeline was implemented:

- **Filtering:** Fields irrelevant to nutritional consultation (e.g., legacy user ratings, internal IDs) were excluded to optimize storage and processing.
- **Validation:** Integrity checks were performed to ensure essential fields (specifically calories and ingredients) were not null. Incomplete records were removed to prevent the RAG system from generating hallucinations regarding nutritional values.

	dish_name	description	ingredients	cooking_method	calories	protein	fat	image_link
0	Mực ống hấp củ đậu	Mực ống hấp củ đậu hấp vừa chín tới, tươi ngon...	Mực ống lớn 3 con: 600g làm sạch không bỏ da, ...	Xào rau củ: Bắc chảo, cho 1 muỗng dầu ăn vào c...	416	16	20	https://monngonmoingay.com/wp-content/uploads/...
1	Chả giò ngũ vị	Chả giò ngũ vị sẽ là món ngon làm chao đảo bữa...	Da bò bía tươi 20 muỗnggiếng, Thịt bò xay 100g...	Đặt da bò bía lên đĩa cho nhân thịt bò vào, xé...	418	29	33	https://monngonmoingay.com/wp-content/uploads/...
2	Canh ba màu	Dù bạn có ăn chay hay không, món ăn Canh Ba Mả...	Khoai mì : 200g, Đậu phộng : 100g, Đậu hũ non ...	Phi thơm hành boaro băm, cho đậu phộng giã vào...	220	33	18	https://monngonmoingay.com/wp-content/uploads/...
3	Tàu hũ ky chiên xá ớt	Một món ăn chay ngon và chi phí thấp, cách chế...	Tàu hũ ky lá: 50g, Chao trắng: 1 hũ, Bột mì: ½...	Chiên miếng tàu hũ ky cho vàng, dùng cọ phết h...	251	12	38	https://monngonmoingay.com/wp-content/uploads/...
4	Gỏi ngó sen tôm thịt	Gỏi ngó sen tôm thịt là một món ăn thường thấy...	Thịt ba chỉ 200g, Tôm sú 200g, Ngó sen non 300...	Nước mắm trộn gỏi: hòa tan 1/2 chén nước mắm, ...	317	28	32	https://monngonmoingay.com/wp-content/uploads/...

Figure 13: Key Data Fields Selected for the RAG System

2.4.2 Semantic Data Engineering

To optimize the retrieval performance of the RAG system, the curated data underwent a rigorous Semantic Data Engineering process. Unlike traditional relational database indexing, Vector Search requires data to be represented as coherent, context-rich textual units rather than isolated attributes. We implemented a two-phase transformation strategy: Contextual Serialization and Metadata Decoupling.

2.4.2.1 Data Cleaning

a. Contextual Serialization

Raw tabular data (CSV) inherently lacks semantic connectivity between columns. For instance, an embedding model treating the ingredients column in isolation might lose the association with the specific dish_name.

To resolve this, we employed a Contextual Serialization technique. We algorithmically constructed a "Unified Document" for each food item by concatenating key semantic fields into a single, narrative-like text block. This ensures that the vector representation captures the holistic relationship between what the dish is (Name/Description), what it contains (Ingredients), and how it is made (Method).

The transformation logic is defined as follows:

- Input: Isolated fields (dish_name, dish_type, description, ingredients, cooking_method).
- Process: Concatenation with explicit semantic labels (e.g., "Món ăn:", "Thành phần:") to guide the Embedding Model's attention mechanisms.
- Output: A dense semantic block optimized for vectorization.

b. Metadata Decoupling Strategy

A critical engineering decision was to separate Semantic Content from Operational Data.

1. Noise Reduction in Vector Space:

Information such as image_link (URLs) and raw numerical IDs contains high entropy but low semantic value for natural language understanding. Including URLs in the embedding text introduces "noise," potentially degrading the quality of vector similarity scores. Therefore, these fields were excluded from the embedding payload.

2. Structured Metadata Injection:

Instead of embedding them, quantitative fields (calories, protein, fat) and visual resources (image_link) were injected as Structured Metadata attached to the Vector Node. This architecture allows the RAG system to perform:

- Hybrid Retrieval: Filtering results based on hard constraints (e.g., calories < 500) while searching for semantic matches.
- Deterministic Rendering: The Android client receives the exact image URL and nutritional values directly from metadata, eliminating the risk of the LLM "hallucinating" or altering these precise figures during text generation.

c. Data Normalization

Before serialization, a normalization layer was applied to ensure consistency:

- Unicode Standardization: Converting Vietnamese character sets to a unified standard (NFC) to prevent encoding errors during tokenization.
- Unit Standardization: Normalizing diverse units (e.g., "kilo calories", "Kcal") into a single format (kcal) and converting text-based numbers into integers to facilitate mathematical operations in the metadata layer.

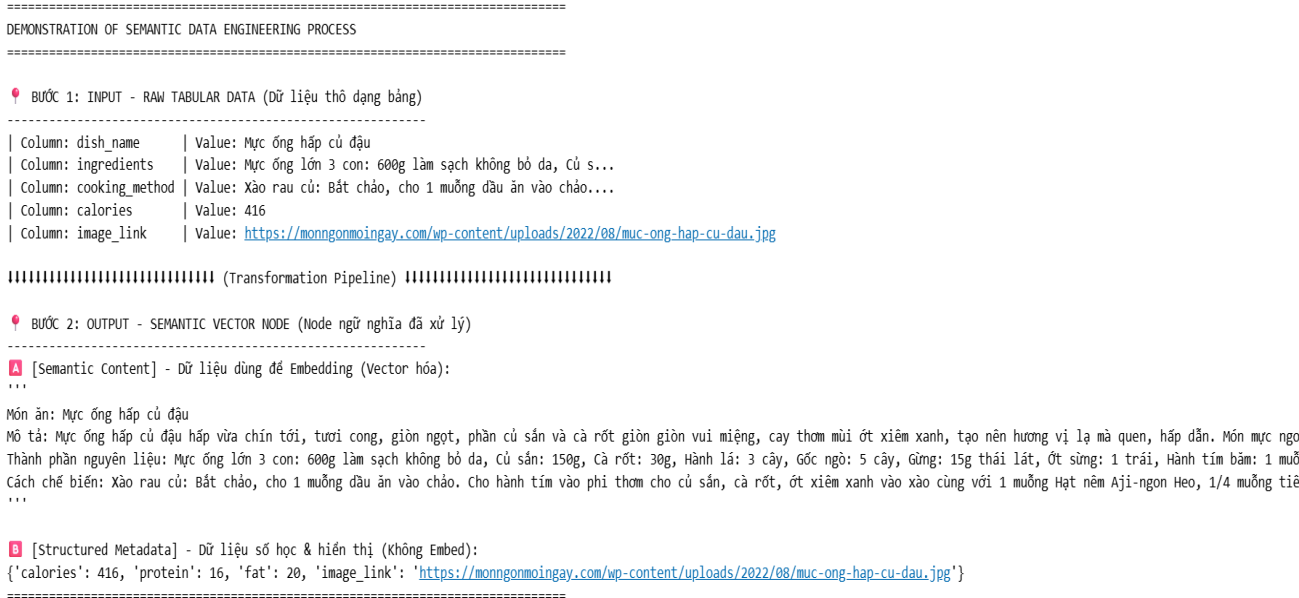


Figure 14: Demonstration of Semantic Data Engineering Process

2.4.3 Chunking Strategy: Row-based Atomic Granularity

Unlike traditional RAG pipelines that process unstructured documents (e.g., PDFs) using fixed-size window chunking (e.g., RecursiveCharacterTextSplitter), this research adopts a Row-based Contextual Chunking strategy.

a. Rationale

Standard chunking methods slice text based on character count (e.g., every 512 tokens). Applying this to culinary data poses a significant risk of **semantic severance**. For instance, a fixed-size splitter might cut a recipe halfway through the "Cooking Method," separating the instructions from the "Ingredients" list. This would prevent the retrieval system from accurately answering complex queries that require both pieces of information simultaneously.

b. Implementation

We defined the "Atomic Unit" of retrieval as a single culinary entity (one dish).

- Granularity: 1 Database Row = 1 Vector Node.
- Coherence: By enforcing this logical boundary, we ensure that every retrieved context window contains the complete, self-contained knowledge regarding a specific dish, including its description, ingredients, and cooking instructions.

- Outcome: This strategy maximizes Context Recall, ensuring that the LLM receives comprehensive information for generation without needing to retrieve multiple fragmented chunks.

2.4.4 Vector Embedding and Storage Architecture

a. Embedding Model Selection

To convert the semantic text nodes into high-dimensional vectors, the system utilizes the **AITeamVN/Vietnamese_Embedding [8]** model.

- Domain Adaptation: Unlike generic multilingual models (e.g., OpenAI text-embedding-3), this model is optimized for the Vietnamese linguistic landscape, effectively capturing the nuances of culinary terminology (e.g., distinguishing between "hầm", "tiềm", and "kho").
- Computational Efficiency: The embedding process is accelerated using NVIDIA CUDA technology, processing the dataset in batches to minimize latency.

b. Vector Database (ChromaDB)

The generated vectors are persisted in **ChromaDB**, an open-source vector store chosen for its lightweight architecture and high performance in local environments.

- Indexing: We utilize Hierarchical Navigable Small World (HNSW) graphs for approximate nearest neighbor (ANN) search.
- Metadata Filtering: ChromaDB is configured to index the structured metadata (calories, image links) separately, enabling the Hybrid Search capabilities discussed in Section 2.4.2.

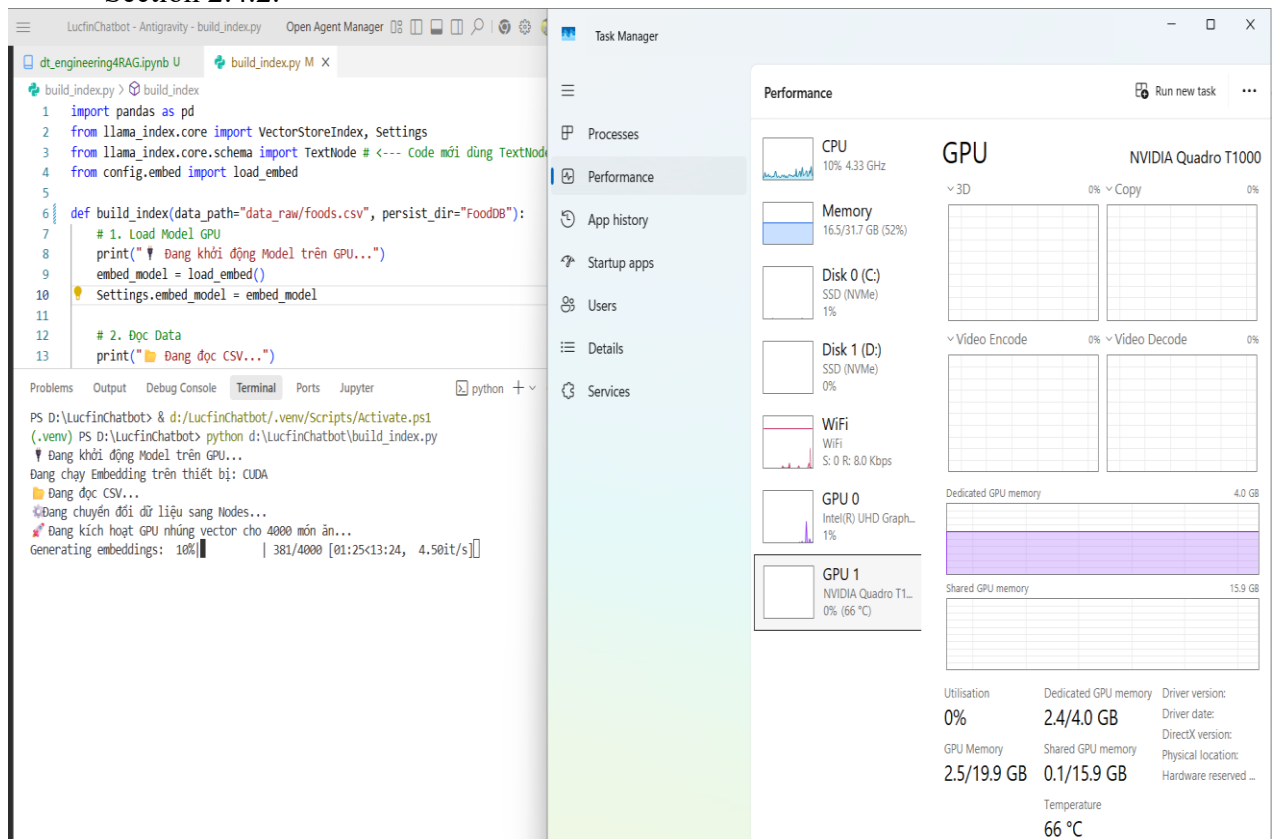


Figure 15: Vector Embedding Process. The system utilizes GPU acceleration to encode processed semantic nodes into the ChromaDB vector store.

2.5 Overall Operational Architecture of the RAG Chatbot

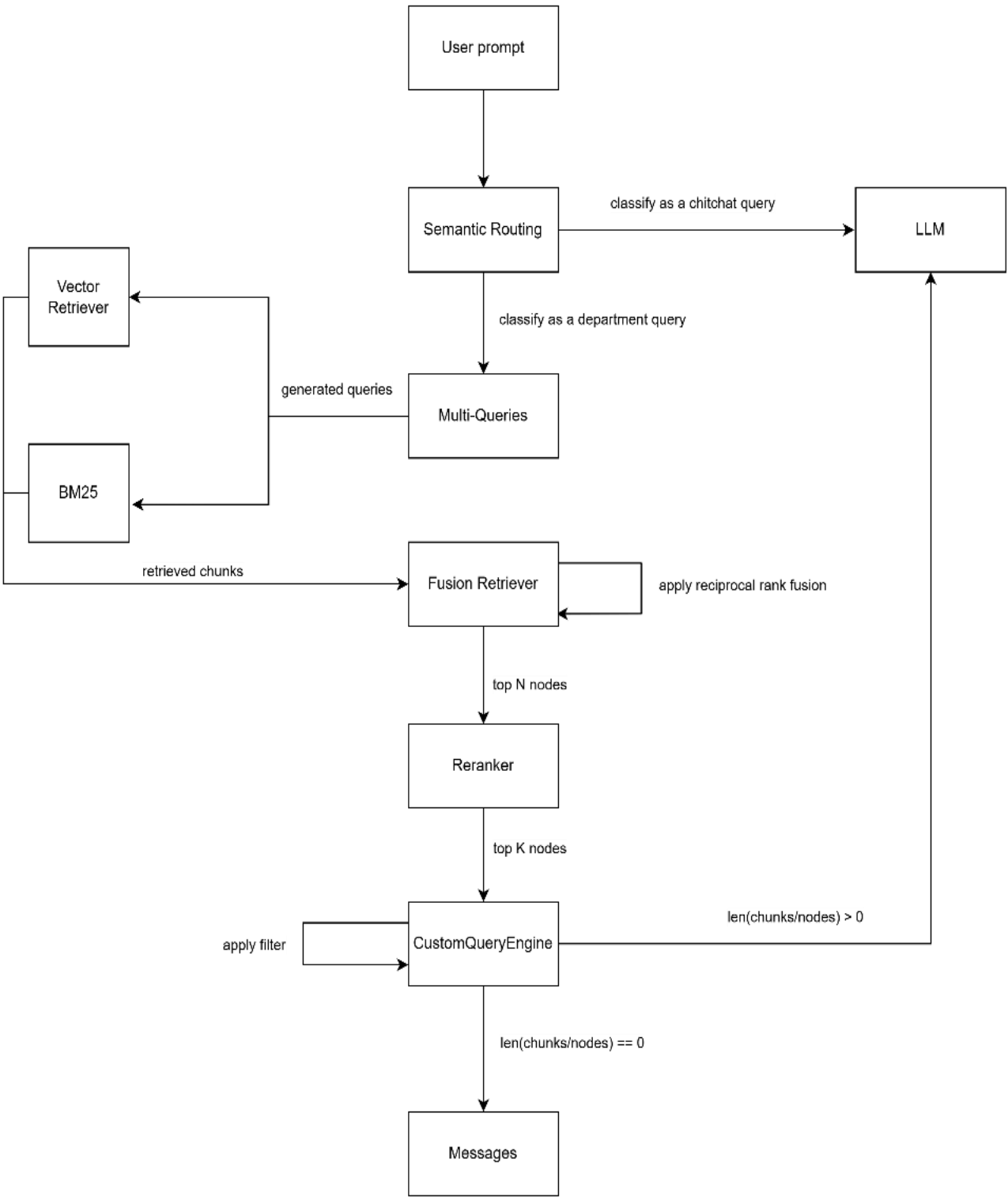


Figure 16: Overall Operational Architecture of the RAG Advance Chatbot Project [4]

2.5.1 RAG Components and Their Roles

Table 4: :RAG Components and Their Roles in Explanation

Component	Role
Runtime environment	Provides the infrastructure and computational resources to host and execute all system components.
Api/endpoints	Serves as the main communication gateway for the entire system. It creates an API endpoint (/ask) to receive questions from the user, then forwards the request to the Router for processing and returns the final result to the interface.
BM25	The BM25Retriever applies the BM25 ranking algorithm for lexical search, retrieving the most relevant documents based on keyword frequency and importance. Unlike embeddings, it relies on exact keyword matching. Here, it is customized for Vietnamese by using a specific tokenizer and disabling stemming to improve accuracy.
Config/ embed	Responsible for "translating" text into computer language. It loads a specialized Vietnamese embedding model (AITeamVN/Vietnamese_Embedding), which converts all text into numerical vectors (embeddings), allowing the computer to understand and compare meanings.
Config/llm	Responsible for initializing and configuring the system's LLMs. It securely reads the API key from the environment file and then establishes a connection to the Large Language Model (qwen/qwen3-32b) via the Groq service to generate the final answer.
Config/vector store	Acts as the system's knowledge repository. It uses ChromaDB to create and manage the vector database, where all vectorized documents are persistently stored on your machine.
CustomQueryEngine	custom_query_engine is designed to implement a specialized query engine that controls how users access and receive answers from the RAG system. Its main purpose is to retrieve, rerank, and filter documents by department, ensuring that employees only see information they are authorized to access. It then generates a prompt for the LLM to produce an answer strictly based on the retrieved content. In short, this module customizes the default query flow of LlamaIndex to enforce access control, accuracy, and relevance in enterprise environments.
Chunker	Splits preprocessed text into smaller, manageable segments (chunks) to optimize embedding and retrieval.
Data preparation pipeline	Coordinates the transformation of raw data into a structured, chunked, and vectorized format suitable for retrieval and generation.
Fusion Retriever	The Fusion Retriever combines results from different retrievers (e.g., BM25 and vector search). Using techniques like reciprocal rank fusion, it merges and re-ranks the results to provide a more accurate and comprehensive set of documents.
Index builder	Acts as the knowledge base organizer. It performs the entire data processing pipeline: reading documents from files, automatically attaching metadata (like department ID), chunking the text, and then vectorizing each chunk into the Vector Store, making it ready for retrieval. [2]
Multi-Queries	Multi-Queries generate multiple variations of the original query with similar meaning. This expands the search coverage and ensures that more relevant documents can be retrieved, even if the wording differs.
Sematic Routing	Semantic Routing classifies the user's query into different categories (e.g., department-related or general chitchat). It ensures the query is directed to the correct processing flow, improving relevance and efficiency.
Reranker	This is a specialized component that improves the accuracy of search results. It acts as a more sophisticated second-stage filter: after the Retriever fetches a

	list of potential documents, the Re-ranker re-evaluates and scores the relevance of each document against the original query, then re-sorts them to select only the most relevant results
Retriever/ custom_query_engine	Acts as the coordinator of the Q&A process. It receives the question, directs the Retriever to search for documents, checks the user's departmental access permissions, and then sends the retrieved information to the LLM to generate the final answer.
Retriever/ custom_retriever	The retriever acts as an information retrieval component within the knowledge base. It searches the Vector Store to find the most relevant documents for a given query, while also implementing post-query filtering logic to check metadata and departmental access permissions.
Router/handler	Acts as the information searcher within the knowledge base. It searches the Vector Store to find the most relevant documents to the question, while also implementing post-query filtering logic to check metadata and enforce departmental access permissions.
Router/parser	Defines separate processing flows for each type of request: one flow for general conversation (chitchat) and another to initialize and execute the RAG pipeline for business-related queries that require document lookup.
Router/prompt	Provides the prompt templates to control the LLM's behavior. It defines the processing options (business query, chitchat) and instructs the LLM to classify the user's question, while also requesting the LLM to return its decision in a structured JSON format.
Router/router	Acts as the central router of the system. This module uses the LLM to classify the user's question into different processing flows (e.g., business or chitchat), then routes the request to the appropriate handler function for execution.
Router/schemas	Defines the data schema for the LLM's classification output. Using Pydantic, it creates a "template" (Answer) to ensure that the output from the LLM is always in a consistent and valid format before it is processed.
Retriever	Searches the vector database for the most relevant chunks in response to a query embedding.
Vector database engine	Stores, indexes, and enables efficient similarity search of vector embeddings.
User interface	Enable users to submit queries and view generated responses, facilitating interaction with the system.
utils	A collection of small, reusable helper functions. They perform auxiliary tasks such as string formatting, data extraction, and cleaning to support other core components, which helps keep the codebase organized and tidy.
evaluation	Responsible for measuring and validating the quality of the Retriever component. This module automatically generates a sample question answer dataset from the source documents, then calculates standard metrics like Hit Rate and MRR to evaluate its information retrieval performance.

CHAPTER 3: DEVELOPMENT OF FOOD RECOGNITION AND NUTRIENT ANALYSIS MODULE

3.1 Data Preparation

To construct a robust dataset capable of facilitating both precise food recognition and granular nutrient estimation, we implemented a **Hybrid Data Pipeline**. This pipeline integrates real-world data mining, spatial annotation via standard tools, and a semantic annotation process powered by Large Vision-Language Models (LVLMs) with a Human-in-the-loop verification mechanism.

The workflow is divided into four distinct stages:

1. **Data Mining:** In-the-wild image collection.
2. **Spatial Annotation:** Bounding box labeling using CVAT.
3. **Semantic Attribute Extraction:** Automated feature extraction using the "Mixture of Experts" approach (Qwen2-VL).
4. **Human-in-the-loop Refinement:** Data cleaning and standardization using a custom Streamlit application.
5. **Exploratory Data Analysis (EDA)**

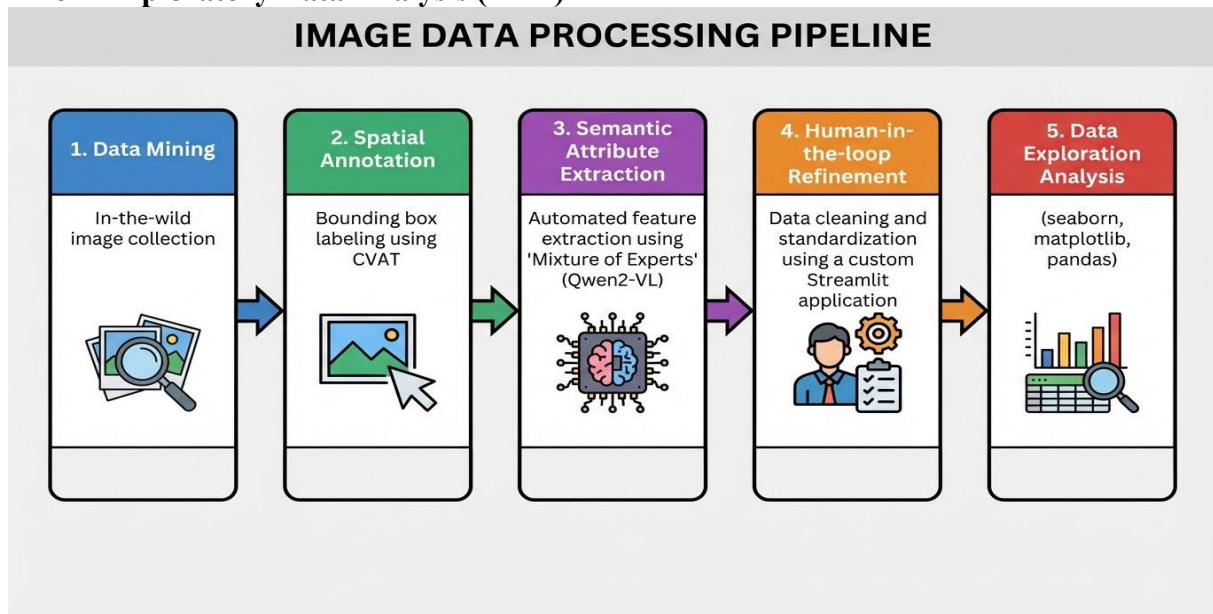


Figure 17: Data Preparation and Processing Pipeline

3.1.1 Data Mining

The image dataset was collected under "in-the-wild" conditions to simulate real-world usage scenarios. The collection process focused on three primary Vietnamese food categories:

Broken Rice Ribs (Suon), Tofu (Dau Hu), and Fried Fish Cake (Cha Ca).

- Acquisition Protocol: Images were captured using smartphone cameras (iPhone/Android) at various local eateries.
- Environmental Variability: The dataset encompasses diverse lighting conditions (natural daylight, artificial warm light), varying viewpoints (top-down, side-angle), and different levels of background complexity.
- Normalization: All raw images were renamed and converted to a standardized JPEG format (e.g., cha_mieng_collected_100.jpg) to ensure consistency across the training pipeline.

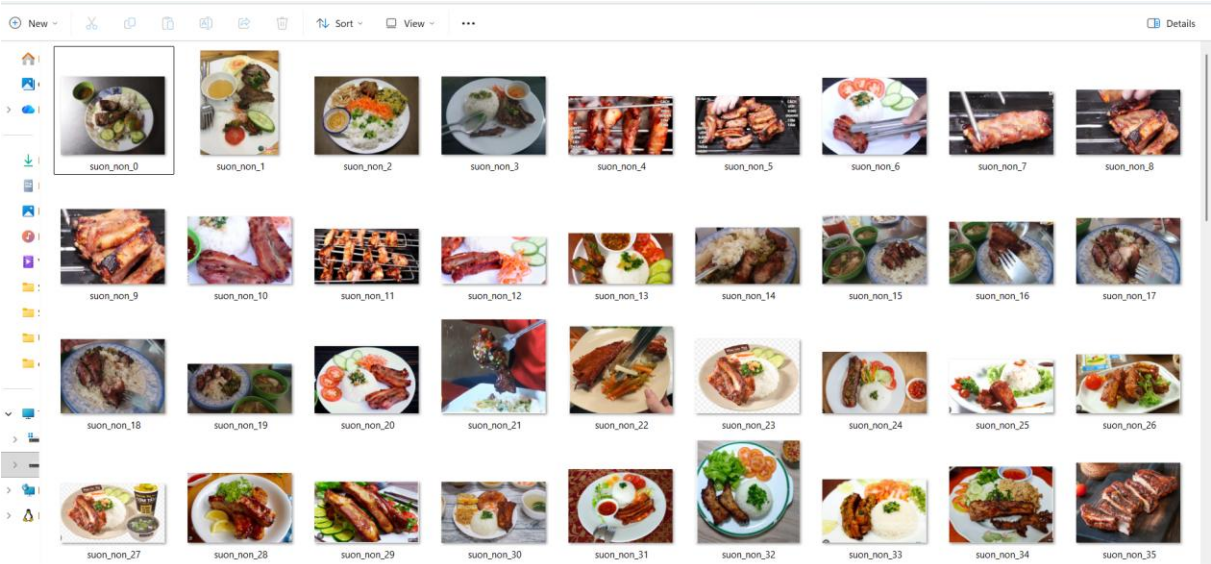


Figure 18: Vietnamese Broken Rice Ribs (Suon)

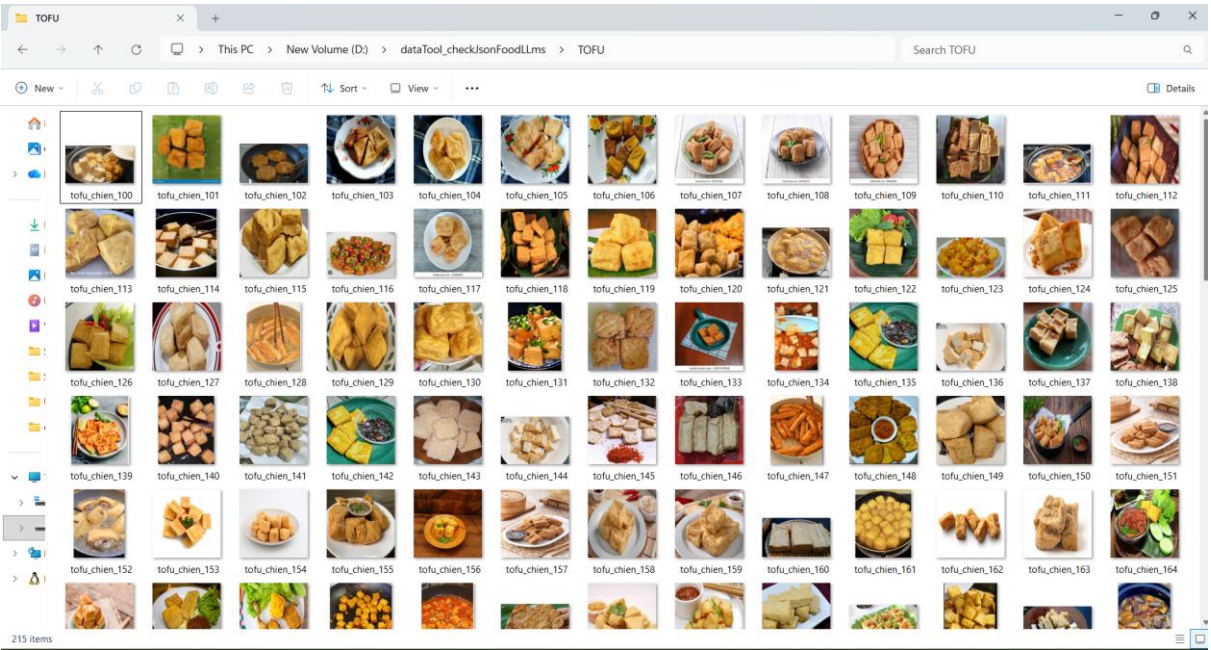


Figure 19: Tofu (Dau Hu)

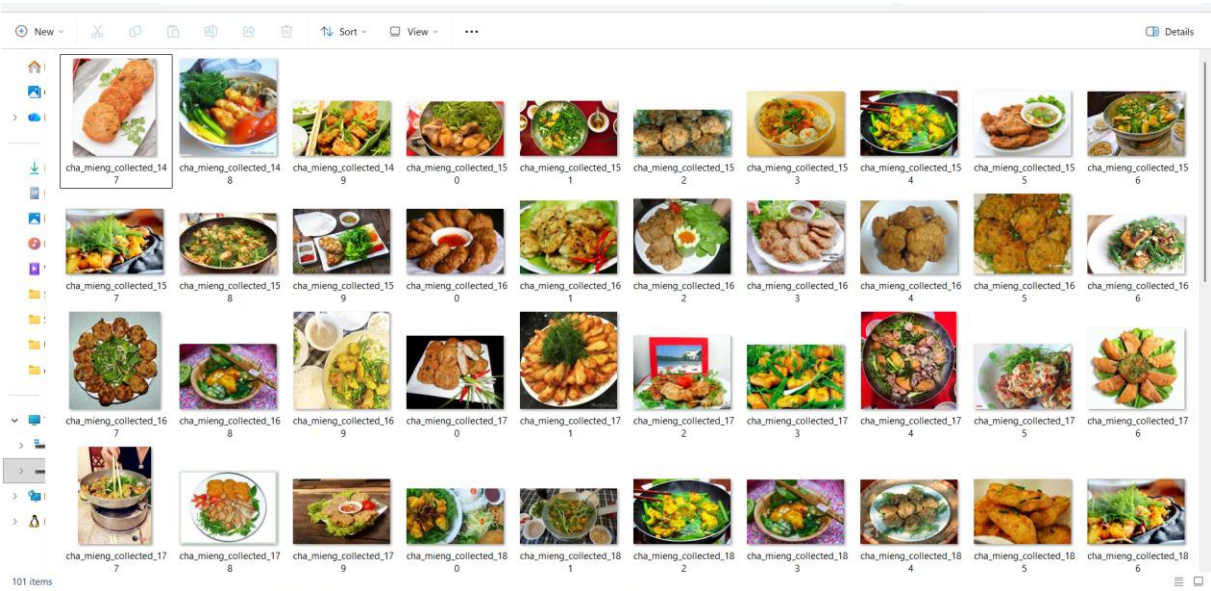


Figure 20: Fried Fish Cake (Cha Ca).

3.1.2 Spatial Labeling with CVAT

For the object detection task (targeting the Faster R-CNN architecture), we employed the Computer Vision Annotation Tool (CVAT) for spatial annotation.

- **Annotation Strategy:** We adopted an Instance-level Labeling approach. Instead of labeling the entire dish as a single object, individual food items were annotated with tight bounding boxes (Rectangles).
- **Objective:** This granularity enables the model to perform counting-based estimation (e.g., detecting 3 separate slices of fish cake), which is crucial for the portion-based nutrient calculation logic defined in the system design.

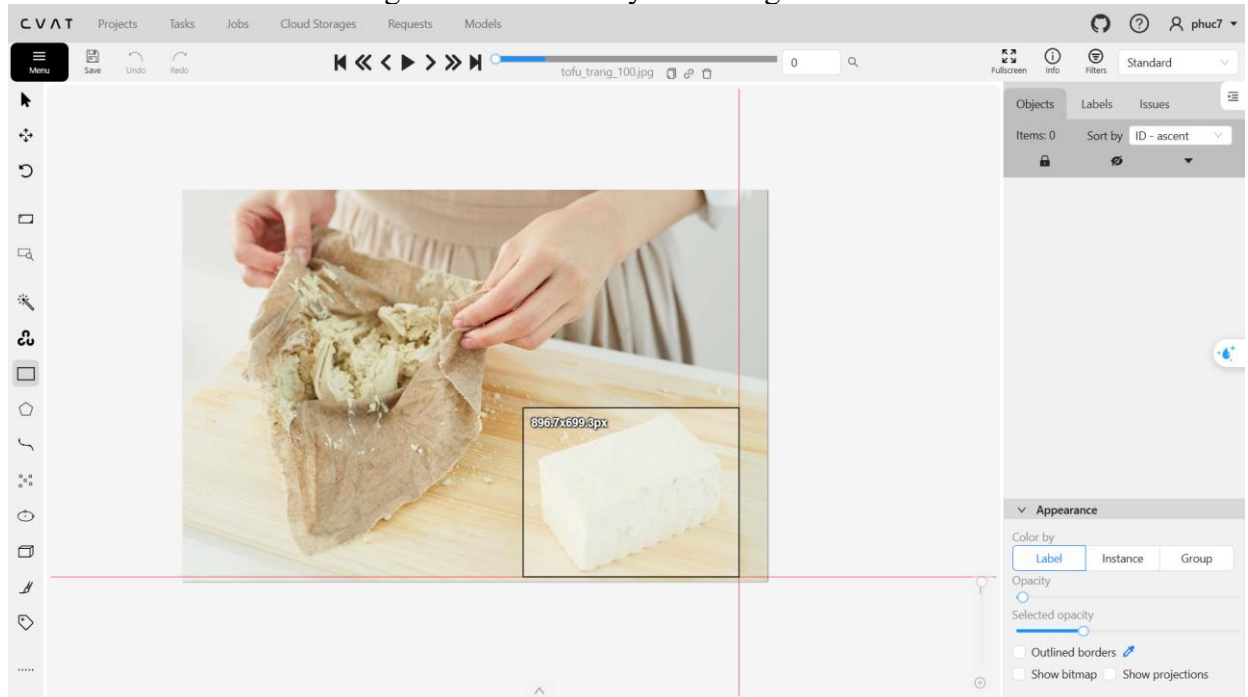


Figure 21: Spatial Labeling with CVAT

3.1.2.1 Data Formatting Standards (COCO Format)

To ensure compatibility with state-of-the-art object detection frameworks, the annotated dataset exported from CVAT was standardized into the **Microsoft COCO (Common Objects in Context)** format.

The data structure is organized as a monolithic JSON file comprising three core components:

- **Images:** Contains metadata for each image file (filename, height, width, and unique ID).
- **Categories:** Defines the mapping between class names (e.g., `suon_cot_let`, `dau_hu_trang`) and their integer IDs.
- **Annotations:** Stores the spatial ground truth for each object instance. The bounding boxes follow the standard format `[x_min, y_min, width, height]`. This standardization allows for seamless integration with the training pipeline and facilitates consistent evaluation metrics (such as COCO mAP).

3.1.2.2 Framework Selection: Detectron2

For the implementation of the Food Recognition module, we selected **Detectron2**, a next-generation library developed by **Facebook AI Research (FAIR)** based on PyTorch.

Justification for Selection:

1. **Modular Architecture:** Detectron2 allows for flexible configuration of network components (Backbone, RPN, ROI Heads), enabling rapid experimentation with different architectures like Faster R-CNN and RetinaNet.
2. **Model Zoo & Transfer Learning:** The platform provides access to a comprehensive "Model Zoo" with high-performance pre-trained weights (typically trained on the COCO 2017 dataset). We leverage **Transfer Learning** by initializing our model with

- a **ResNet-50-FPN backbone**, significantly reducing training time and improving convergence on our custom food dataset.
- 3. **Efficiency:** Detectron2 is highly optimized for GPU acceleration, supporting efficient training loops and fast inference speeds required for real-time nutrient estimation applications.

3.1.3 Automated Visual Attribute Extraction (Mixture of Experts)

To enhance the dataset with semantic depth, such as texture, morphology, and cooking variance, we deployed an AI-Assisted Annotation pipeline. We utilized **Qwen2-VL-7B 16.6GB on local Disk**, a state-of-the-art Vision-Language Model, acting as a "Virtual Nutrition Expert" to automatically analyze images and extract visual features.



```

1 import torch
2 from transformers import Qwen2VLForConditionalGeneration, AutoProcessor
3 from qwen_vl_utils import process_vision_info
4 import json
5 import os
6 from tqdm import tqdm
7
8 # --- CẤU HÌNH ---
9 IMAGE_FOLDER = "/content/test_suan" # Đường dẫn folder ảnh trên Colab
10 OUTPUT_FILE = "/content/train_comtam_qwen.json"
11
12 # --- LOAD MODEL (Tối ưu cho L4 GPU) ---
13 # L4 hỗ trợ Flash Attention 2 -> Tốc độ cực nhanh
14 print("🔄 Đang load Qwen2-VL-7B...")
15 model = Qwen2VLForConditionalGeneration.from_pretrained(
16     "Qwen/Qwen2-VL-7B-Instruct",
17     torch_dtype=torch.bfloat16,
18     attn_implementation="sdpa", # <-- Quan trọng: Dùng cái này để không bị lỗi thiếu thư viện
19 )
20 processor = AutoProcessor.from_pretrained("Qwen/Qwen2-VL-7B-Instruct")
21
22 # --- XỬ LÝ DỮ LIỆU ---
23 images = os.listdir(IMAGE_FOLDER)
24 for image in tqdm(images):
25     image_path = os.path.join(IMAGE_FOLDER, image)
26     processor.load_image(image_path)
27     prompt = "Hãy mô tả chi tiết món ăn trong ảnh, bao gồm các đặc điểm hình thái, kết cấu bề mặt, và các thành phần đi kèm."
28     response = model.generate(prompt, processor.get_prompt_token_types(), max_new_tokens=100)
29     output = processor.decode(response)
30     data = {"id_anh": image, "attributes": output}
31     with open(OUTPUT_FILE, 'a') as f:
32         json.dump(data, f)
33         f.write('\n')
34
35 print(f"Đã xử lý xong {len(images)} ảnh. Kết quả lưu tại {OUTPUT_FILE}")

```

PS D:\cha_ca> & d:/cha_ca/.venv_new/Scripts/Activate.ps1
(.venv_new) PS D:\cha_ca> python d:\cha_ca\check_qwen7b_json.py
🔄 Đang load Qwen2-VL-7B...
Unrecognized keys in "rope_parameters" for "rope type='default': {'mrope_section':
Downloading (incomplete total...): 12%
Fetching 5 files: 0%

Figure 22: Automated Visual Attribute Extraction using Qwen2-VL-7B 16.6GB

Process: The model processes each image and generates a structured JSON output containing detailed visual attributes. Sample AI Output:

```

[
  {
    "Food_Attributes": {
      "Morphology": "Block_Cube",
      "Surface_Texture": "Matte_Porous",
      "Dish_Variant": "Sauce_Topping_Complex"
    },
    "Image_Attributes": {
      "Lighting_Condition": "Natural_Daylight",
      "Background_Complexity": "Clean_Solid",
      "Occlusion_Level": "None"
    },
    "id_anh": "tofu_trang_108.jpg"
  }
]

```

3.1.4 Human-in-the-loop Refinement (Streamlit Tool)

While LVLMS are powerful, they are prone to hallucinations or formatting inconsistencies (e.g., inconsistent key casing). To ensure Data Hygiene, we implemented a Human-in-the-loop validation step using a custom-built Streamlit application.

- **Auto-Mapping:** The tool automatically normalizes JSON keys (e.g., mapping food_attributes to Food_Attributes).

- **Visual Verification:** Annotators review the AI-generated attributes alongside the source image.
- **Rapid Correction:** Errors in texture or variant classification (e.g., mistaking "Fried" for "Steamed") are corrected via a user-friendly UI before final storage.

```
app.py  X  train_tofu_qwen_v3.json  data_exploration.ipynb
app.py > main
1 import streamlit as st
2 import json
3 import os
4 from PIL import Image
5
6 # --- CẤU HÌNH ---
7 IMAGE_FOLDER = "./TOFU"
8 JSON_FILE = "train_tofu_qwen_v3.json"
9

Problems  Output  Debug Console  Terminal  Ports  Jupyter

PS D:\dataTool_checkJsonFoodLLMs> & d:/dataTool_checkJsonFoodLLMs/.venv/Scripts/Activate.ps1
(.venv) PS D:\dataTool_checkJsonFoodLLMs> python -m streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.100.177:8501

2025-12-10 23:35:07.196 Please replace `use_container_width` with `width`.
`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2025-12-10 23:35:12.288 Please replace `use_container_width` with `width`.
`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2025-12-10 23:35:12.427 Please replace `use_container_width` with `width`.
`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2025-12-10 23:35:15.398 Please replace `use_container_width` with `width`.
`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
```

Figure 23: Run Tool App Streamlit

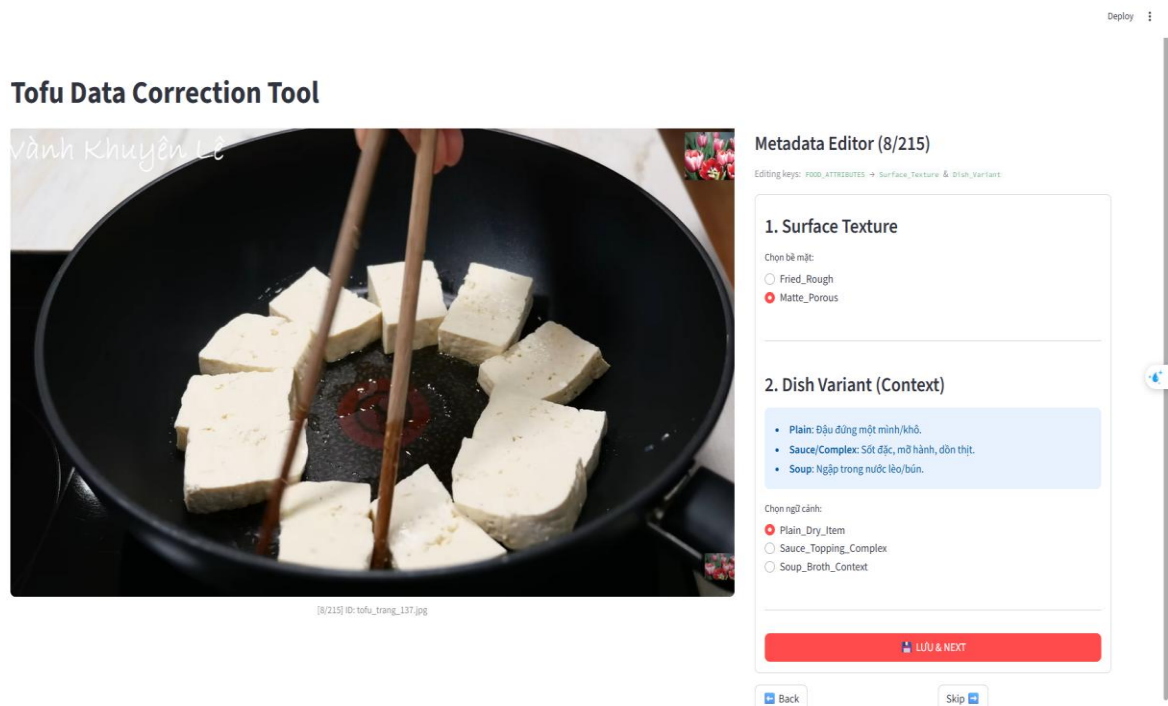


Figure 24: Streamlit Tool for Correction Json File

3.1.5 Exploratory Data Analysis (EDA)

A. Top Features Summary - Environmental Attribute Distribution

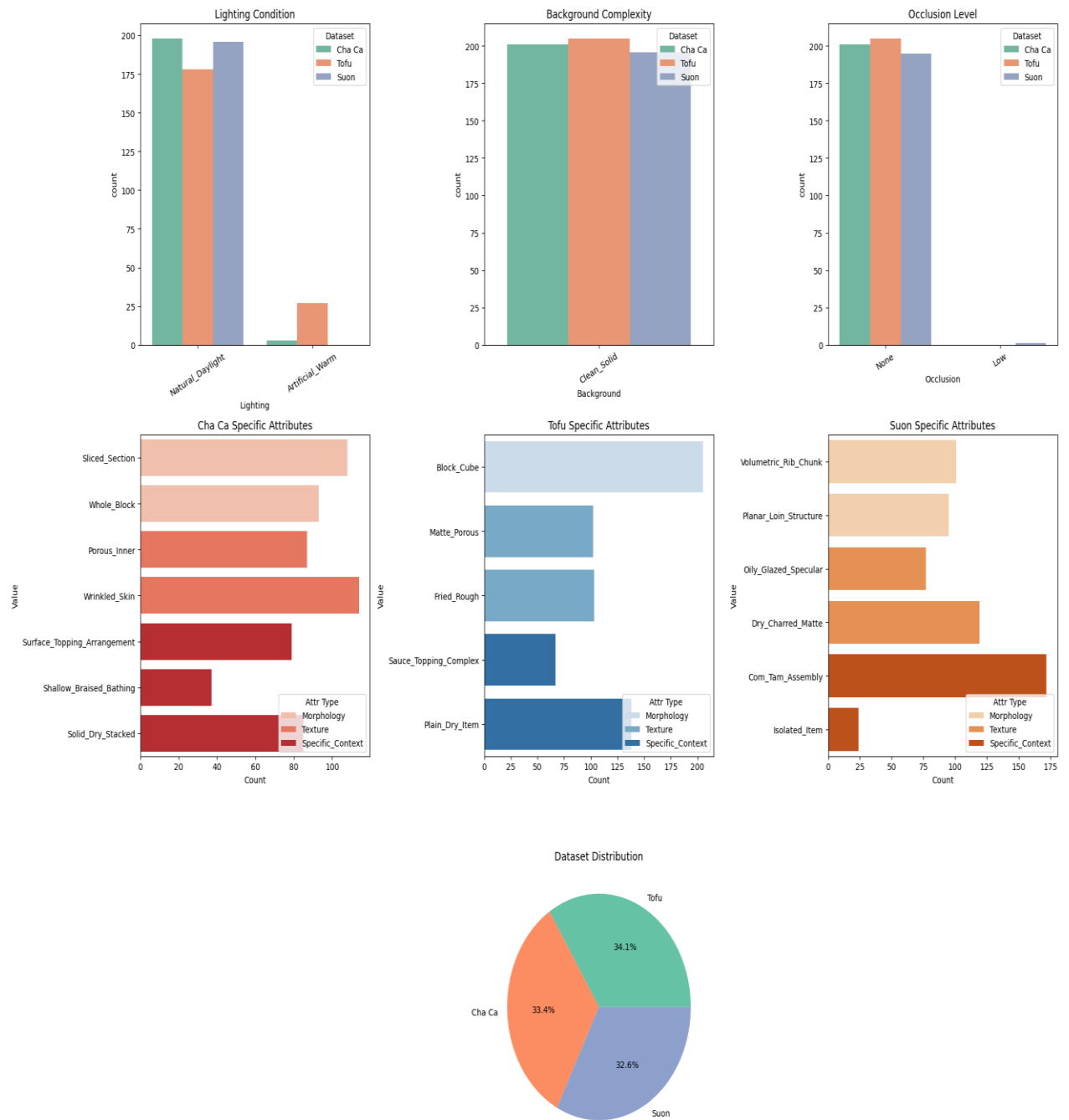


Figure 25 Top Features Summary - Environmental Attribute Distribution

- **Lighting Condition:** The majority of the dataset (>175 samples per class) was captured under "Natural_Daylight", reflecting typical lunch hours. A significant portion of the Tofu dataset also includes "Artificial_Warm" lighting, ensuring the model learns color invariance.
- **Background Complexity:** The dominance of "Clean_Solid" backgrounds helps the model focus on feature extraction during the early training epochs.
- **Occlusion Level:** Most samples are labeled as "None" (low occlusion). While beneficial for initial detection stability, future iterations may require more high-occlusion samples to improve robustness against cluttered food trays.

B. Food-Specific Attribute Analysis

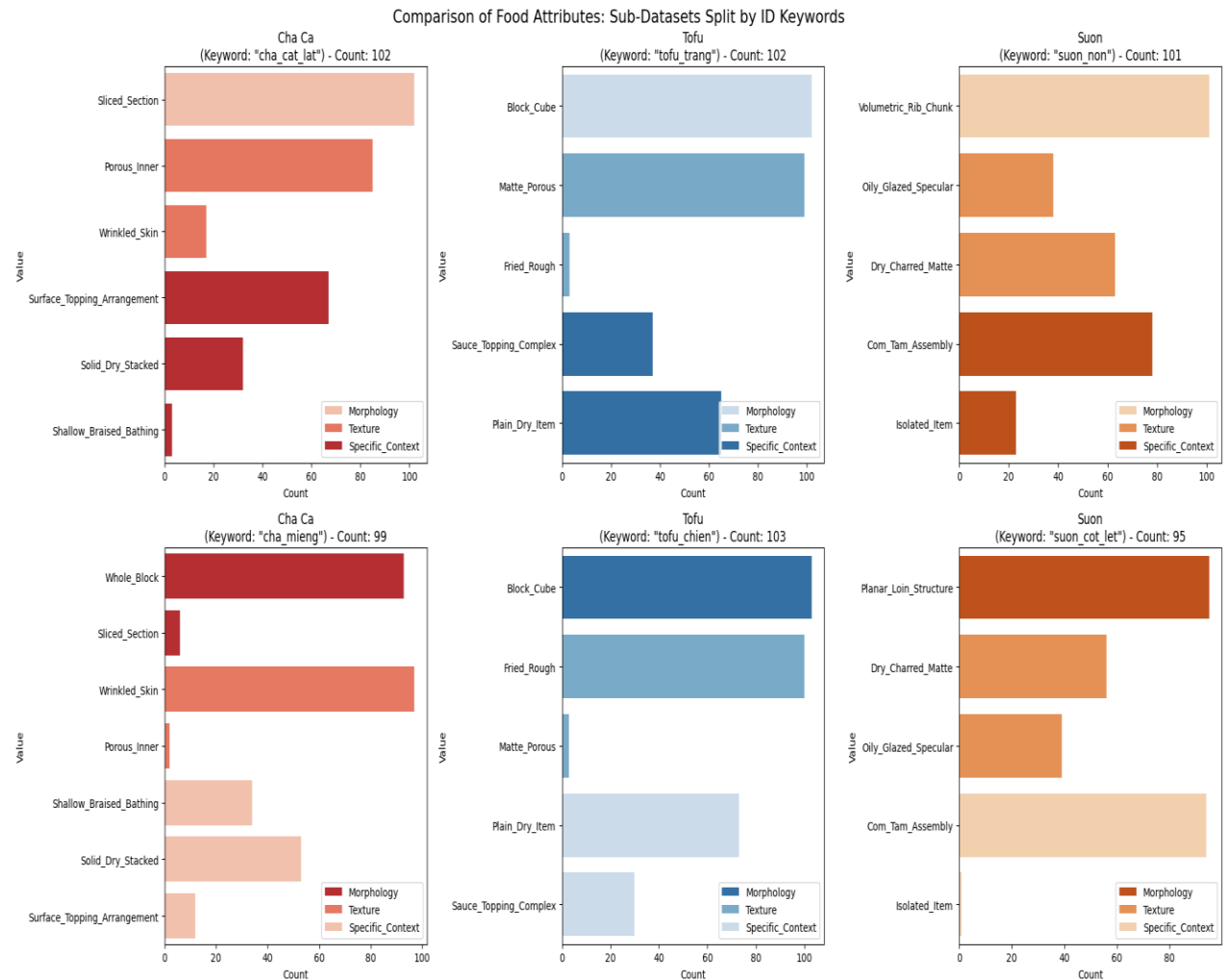


Figure 26: Food-Specific Attribute Analysis

The attribute distribution analysis reveals distinct visual signatures for each food category:

- Fish Cake (Cha Ca):**
 - Morphology:** The dataset is well-balanced between "Sliced_Section" (slices) and "Whole_Block" (whole cakes), covering both serving styles.
 - Texture:** High frequency of "Wrinkled_Skin" and "Porous_Inner" distinguishes fried fish cake from other smooth-surfaced sausages.
- Tofu:**
 - Morphology:** Predominantly "Block_Cube", providing a strong geometric prior for the detection model.
 - Texture:** A clear separation exists between "Matte_Porous" (White Tofu) and "Fried_Rough" (Fried Tofu), which is critical for accurate calorie estimation (as fried items have higher caloric density).
- Pork Ribs (Suon):**
 - Morphology:** The dataset captures both "Volumetric_Rib_Chunk" (Suon non) and "Planar_Loin_Structure" (Cot let).
 - Texture:** The attribute "Dry_Charred_Matte" is prevalent, serving as a key visual cue for grilled dishes.
 - Context:** The "Com_Tam_Assembly" context is dominant, confirming that the model is learning the spatial relationship between the meat and the rice plate.

3.2 Proposed Model Architecture (Faster R-CNN)

We employ the **Faster R-CNN** (Region-based Convolutional Neural Network) architecture, a two-stage object detector known for its high accuracy in locating small and overlapping objects (e.g., tofu cubes in soup).

The architecture consists of two main modules:

1. **Region Proposal Network (RPN):** A fully convolutional network that scans the image to generate "proposals" — regions likely to contain food items.
2. **Fast R-CNN Detector:** A downstream network that extracts features from these proposals (using ROI Align) to perform two tasks simultaneously:
 - **Classification:** Predicting the food category (e.g., cha_ca_lat).
 - **Regression:** Refining the bounding box coordinates to tightly encapsulate the food item for accurate portion estimation.

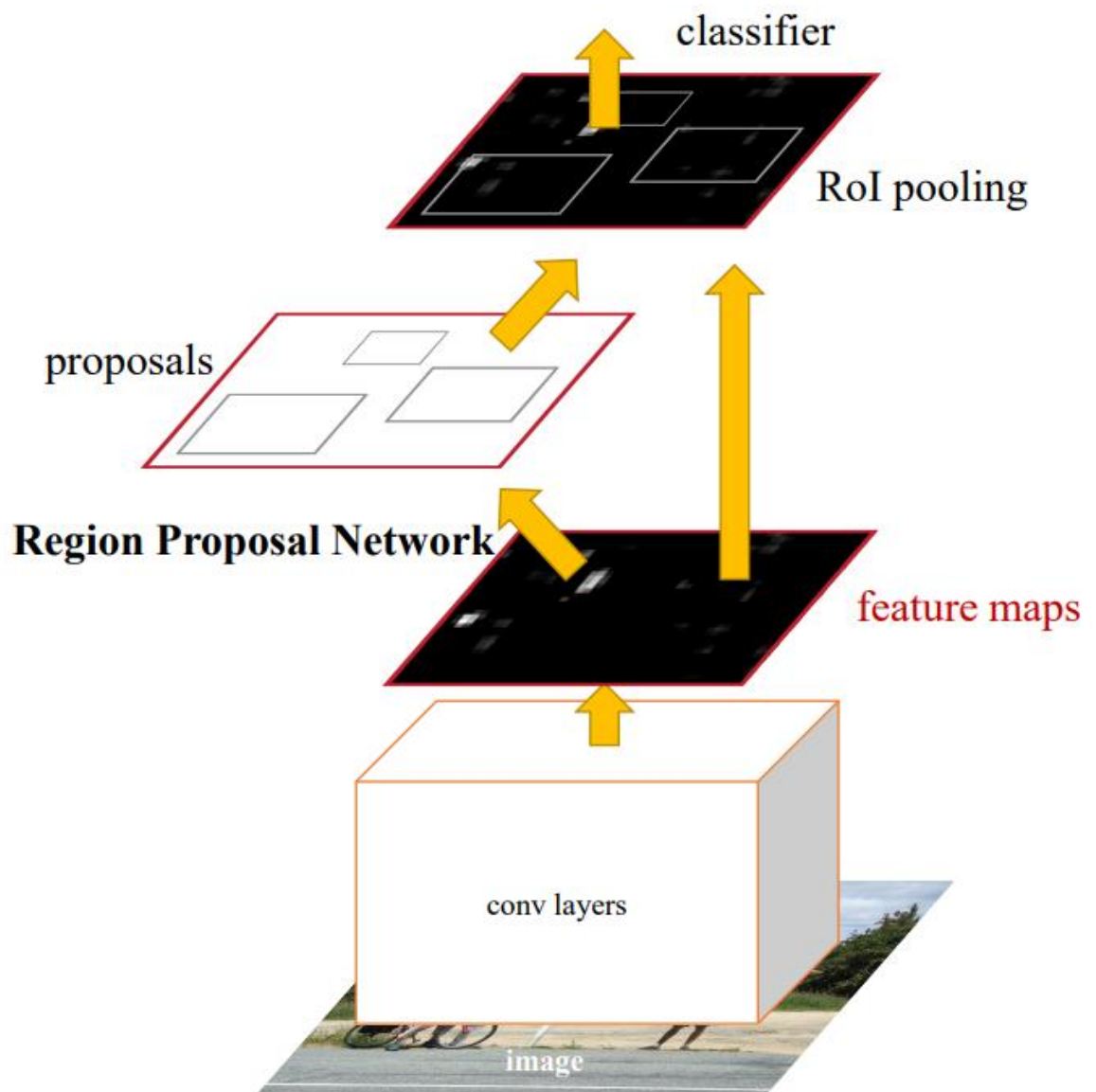


Figure 27: Faster R-CNN is a single, unified network for object detection. The RPN module serves as the ‘attention’ of this unified network. [11]

CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

4.1 Functional Hierarchy Diagram

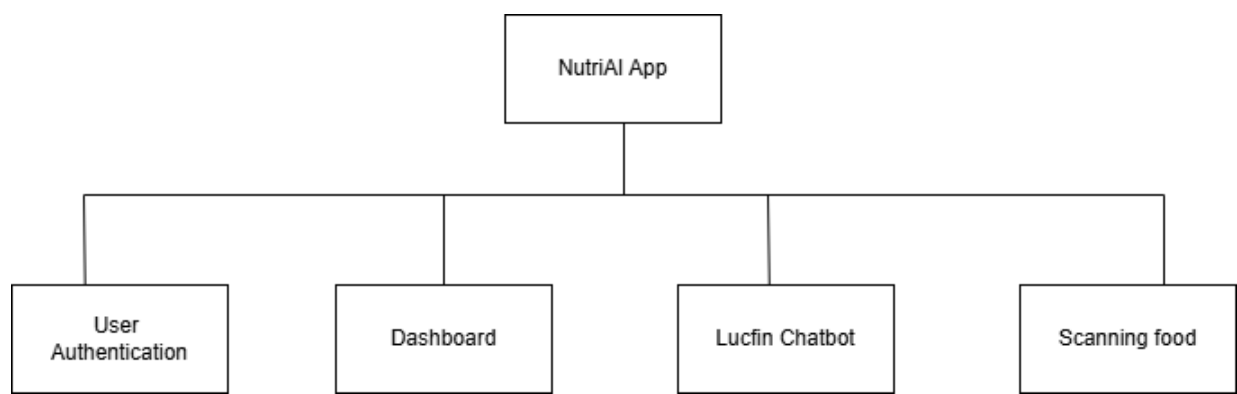


Figure 28: Functional hierarchy diagram

4.2 Functional Decomposition Diagram

4.2.1 User Authentication

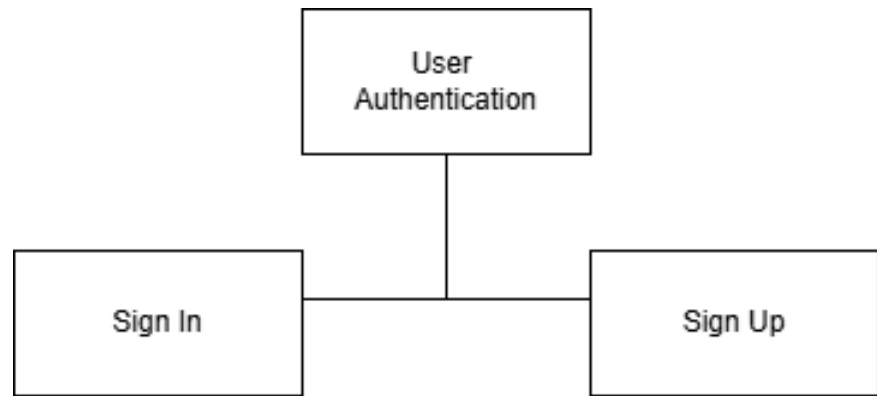


Figure 29: User authentication

Detail description:

The diagram depicts the high-level architecture of the User Authentication subsystem, acting as the secure gateway for the application. It follows a hierarchical decomposition model where the main process is divided into two distinct workflows:

- User Authentication (Root Node): This module orchestrates the identity verification process. It serves as the security layer ensuring that unauthenticated requests cannot reach protected API endpoints (e.g., the LLM inference engine).
- Sign In: Handles the authentication flow for returning users. This component is responsible for validating credentials and issuing secure access tokens (e.g., JWT) for session management.
- Sign Up: Manages the registration flow for new users. This component handles data input validation, password hashing, and the creation of new user entities within the database.

4.2.2 Dashboard

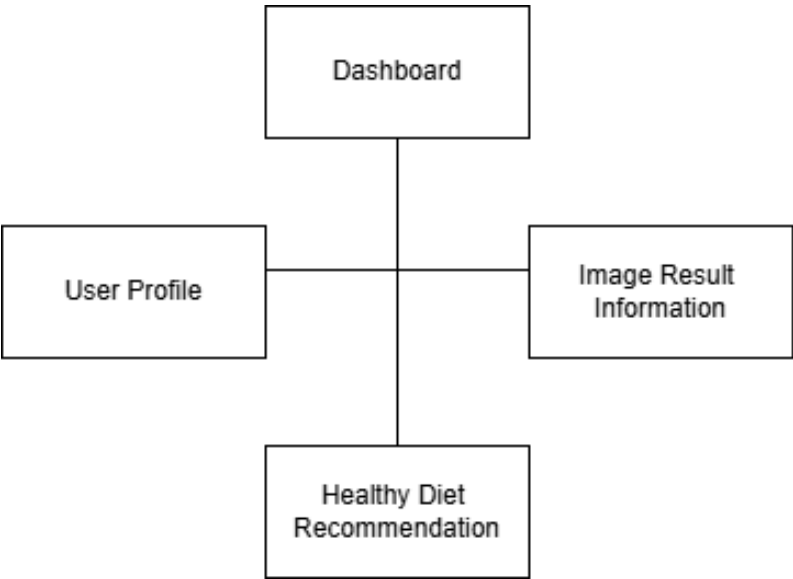


Figure 30: Dashboard

Detail description: The Dashboard functions as the central user interface, aggregating three core operational capabilities:

- User Profile: Enables users to access and review their personal information and health metrics directly on the interface.
- Image Result Information: visualizes the history of food images processed by the Computer Vision model, allowing users to track previously detected items.
- Healthy Diet Recommendation: Features an automated RAG engine that proactively messages the user. It generates personalized dish suggestions, calculates nutritional values, and provides detailed cooking instructions (recipes) based on the user's context.

4.2.3 Scanning Food

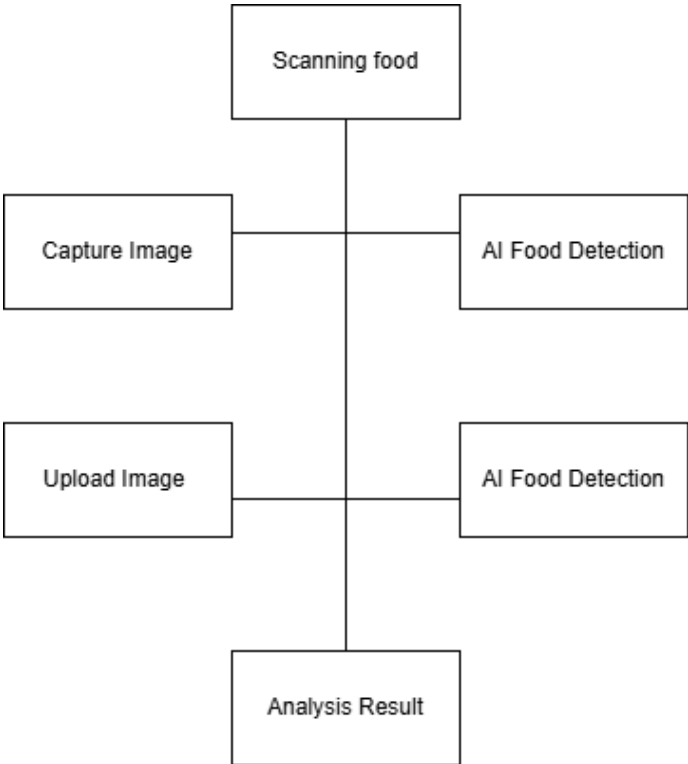


Figure 31: Scanning Food AI

Detail description: Food Scanning & Detection Workflow

Phase 1: Input Acquisition

- The Scanning food process triggers via two entry points:
- Capture Image: Real-time image acquisition via the camera sensor.
- Upload Image: Static image ingestion from local storage/gallery.

Phase 2: AI Inference

- Both input streams feed into the AI Food Detection module. This is the core logic where Computer Vision models (e.g., CNNs, YOLO, or Vision Transformers) perform feature extraction and object classification.

Phase 3: Result Generation

- The pipeline concludes with the Analysis Result, which outputs the predicted class (food name) and potentially associated data (nutrition info, confidence score).

4.2.4 Lucfin Chabot

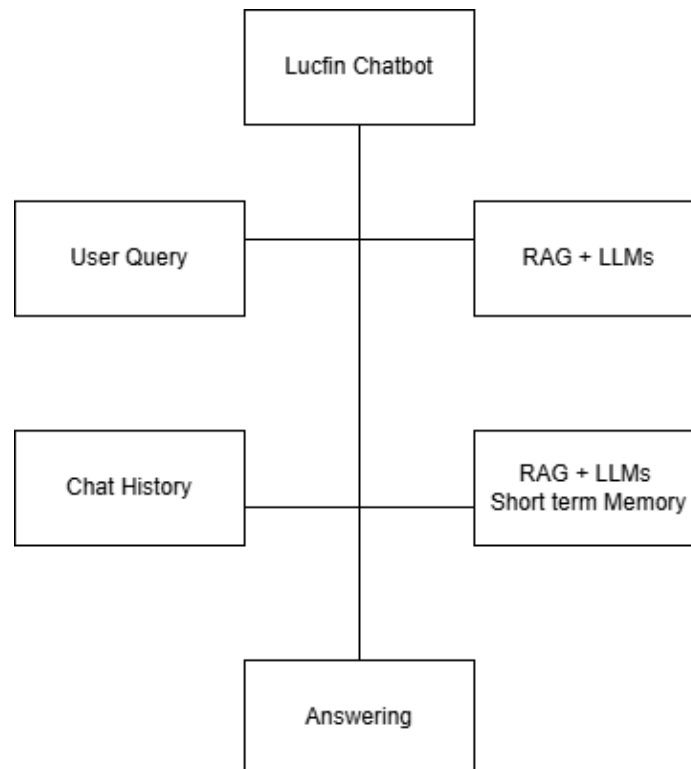


Figure 32: Lucfin RAG Chatbot

Detail description: Conversational RAG Workflow (The Core Logic):

Phase 1: Context Ingestion

The system initiates by aggregating two critical inputs:

- User Query: The immediate prompt or question.
- Chat History: The conversational buffer. This is essential for coreference resolution (e.g., understanding that "it" refers to a previous topic).

Phase 2: RAG & Memory Orchestration (The Engine)

This is the "secret sauce" of the architecture:

- RAG + LLMs: The system performs Semantic Search to retrieve relevant knowledge base chunks based on the user query.
- Short-term Memory: Simultaneously, the LLM processes the retrieved data alongside the Conversation Buffer Memory. This ensures the bot maintains context and continuity across multiple turns, rather than treating every query as an isolated event.

Phase 3: Response Generation

The Answering module synthesizes the final output by combining:

- The retrieved facts (Grounding).
- The conversation history (Context).

- The LLM's generative capabilities (Natural Language).

4.3 Use Case Diagram

4.3.1 Use case for User

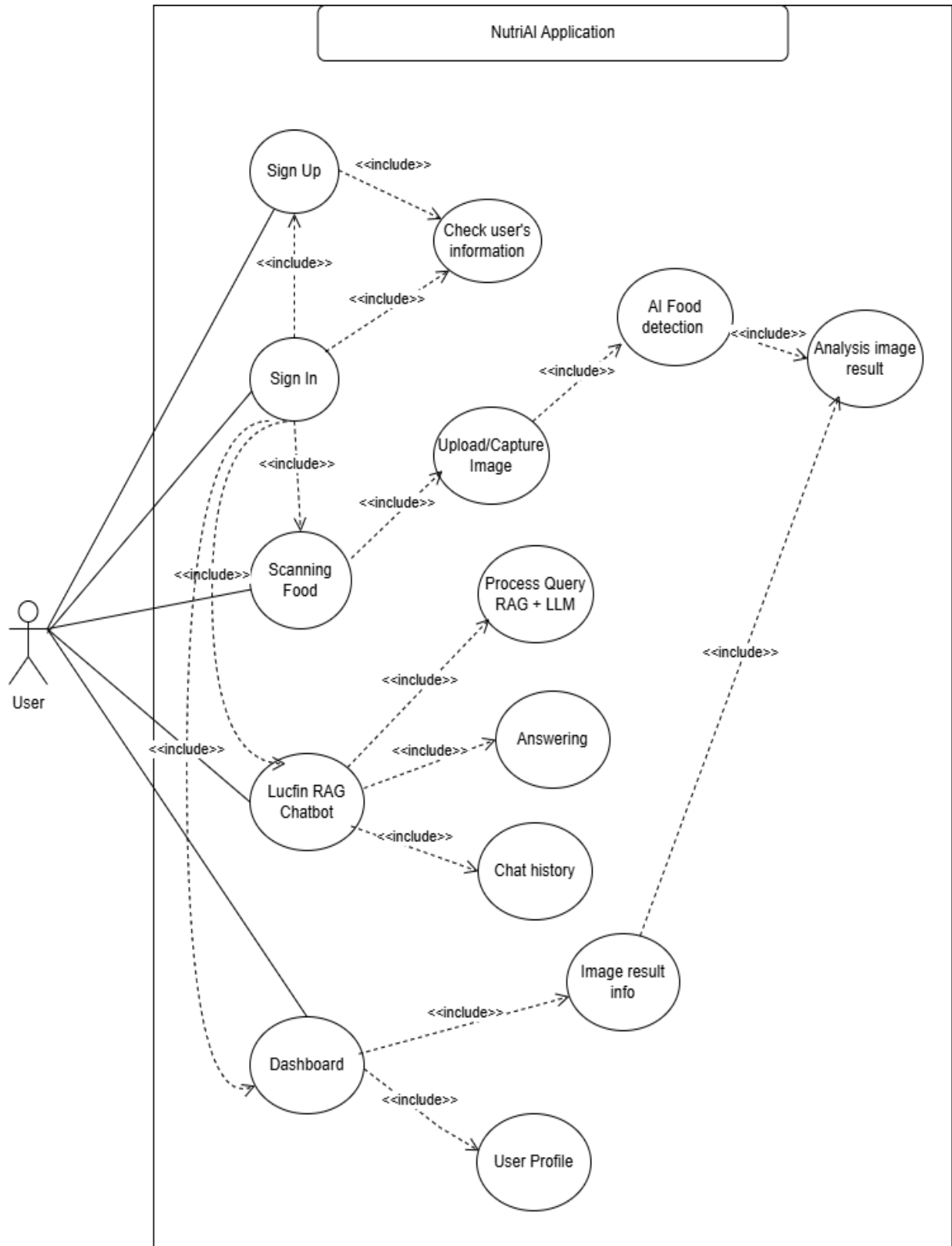


Figure 33: Use-case for User

CHAPTER 5: APPLICATION DEVELOPMENT

5.1 SplashScreen and Onboarding Screen

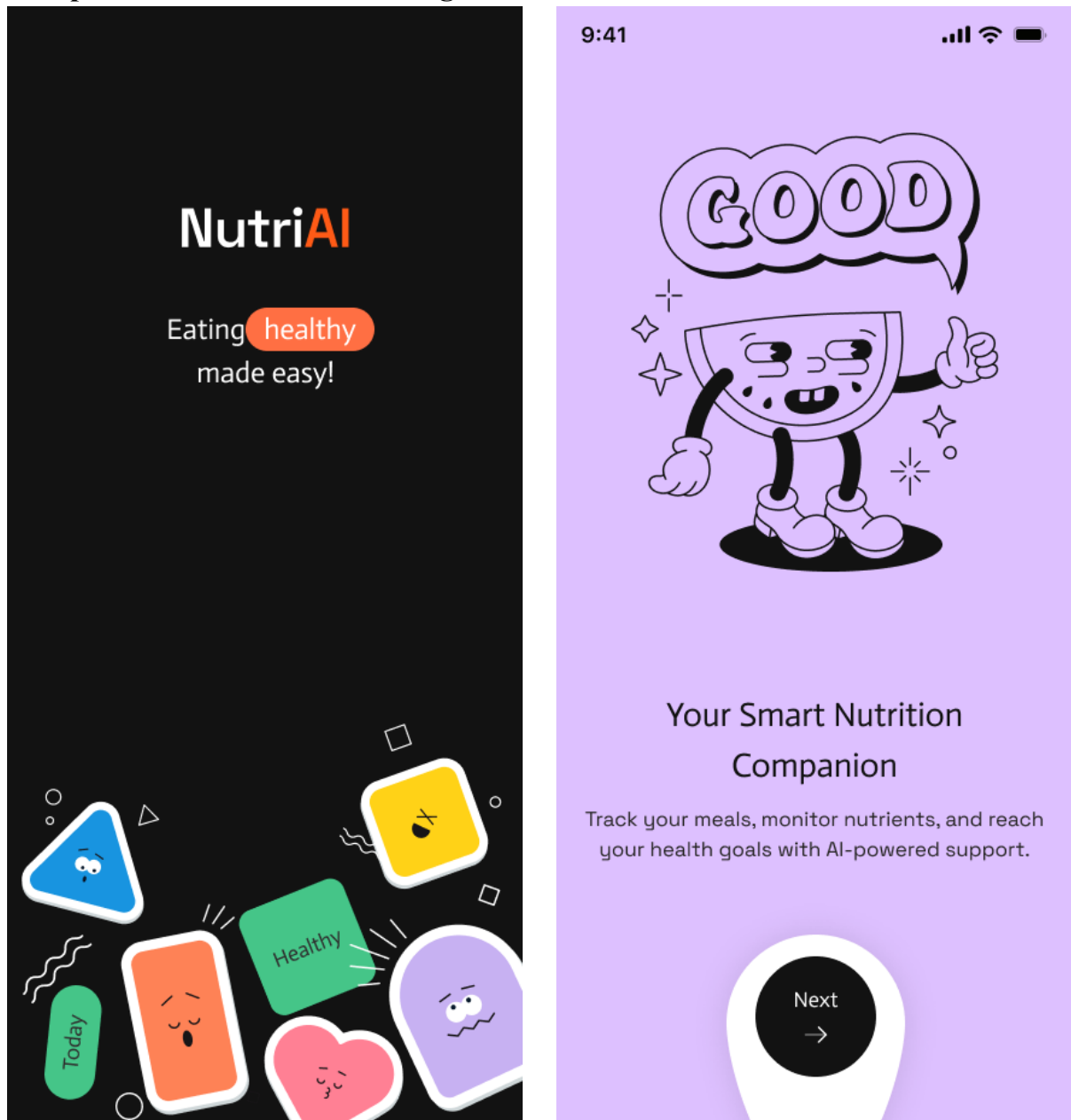


Figure 34: SplashScreen and Onboarding Screen

5.2 Login and Register Screen

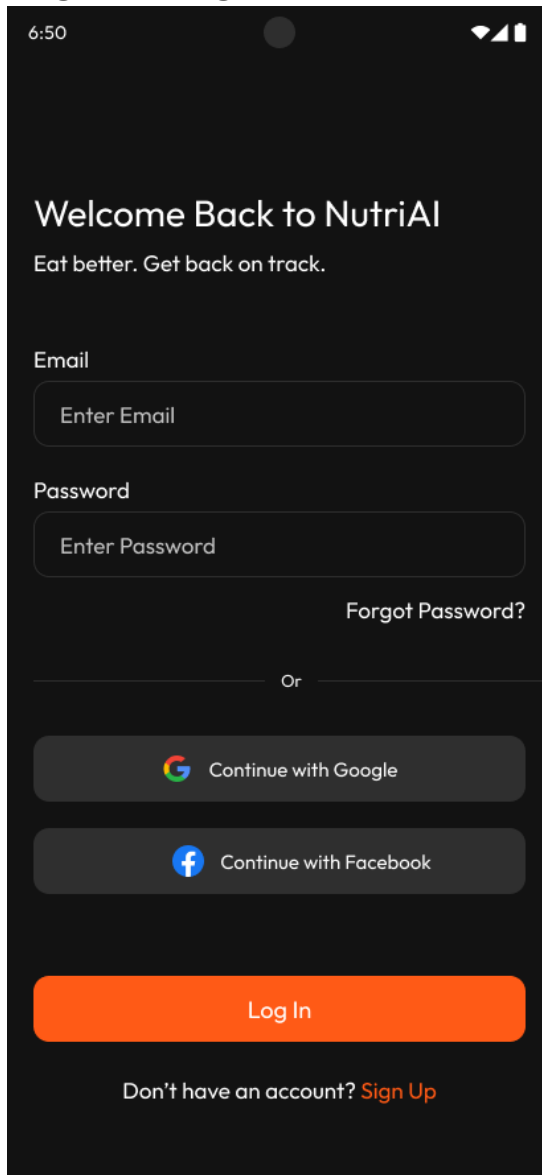


Figure 35: Login Screen

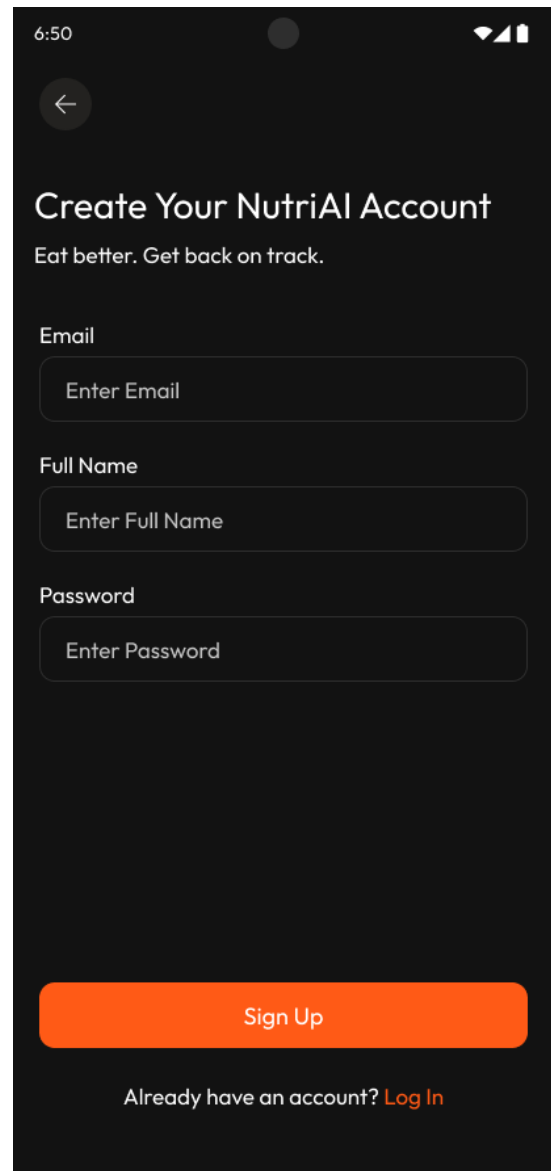


Figure 36: Register Screen

5.3 Dashboard and Add-meal Screen

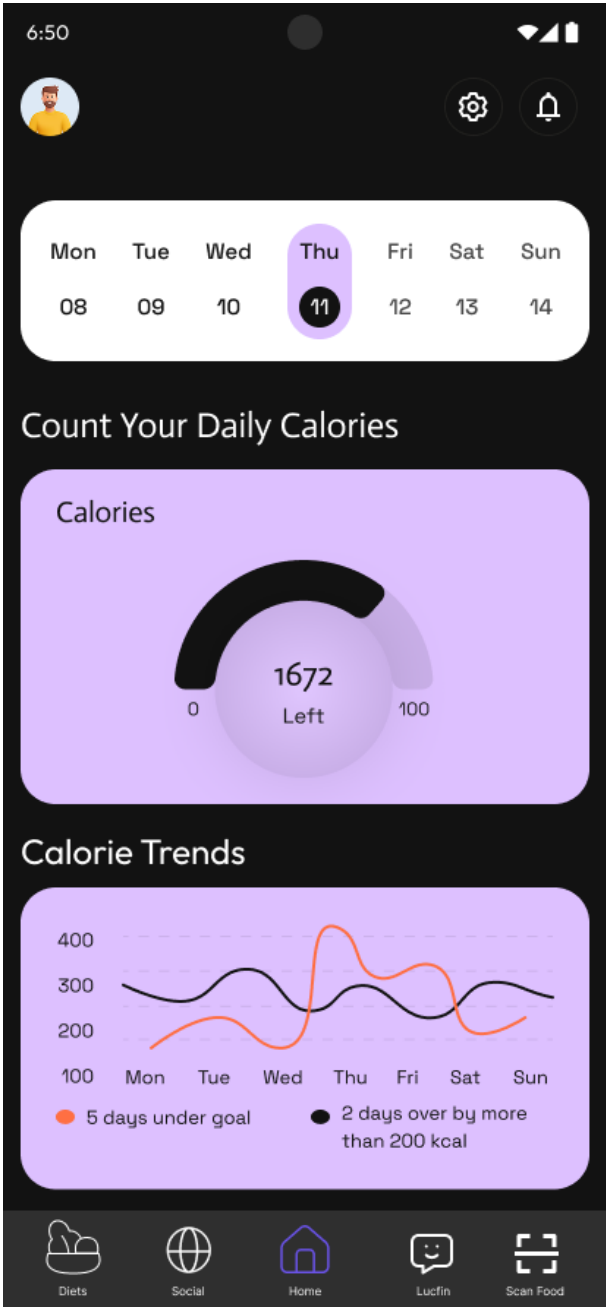


Figure 37: Dashboard Screen

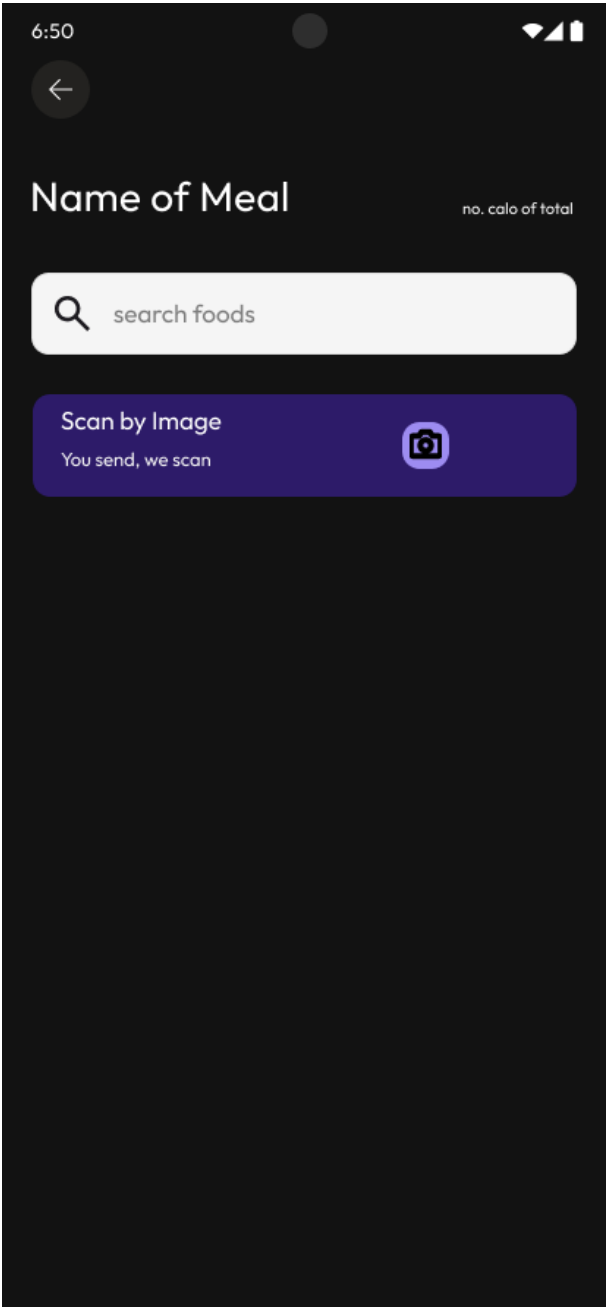


Figure 38: Add-meal Screenshot

5.4 Scanning food and Scanning food Rresult Screen

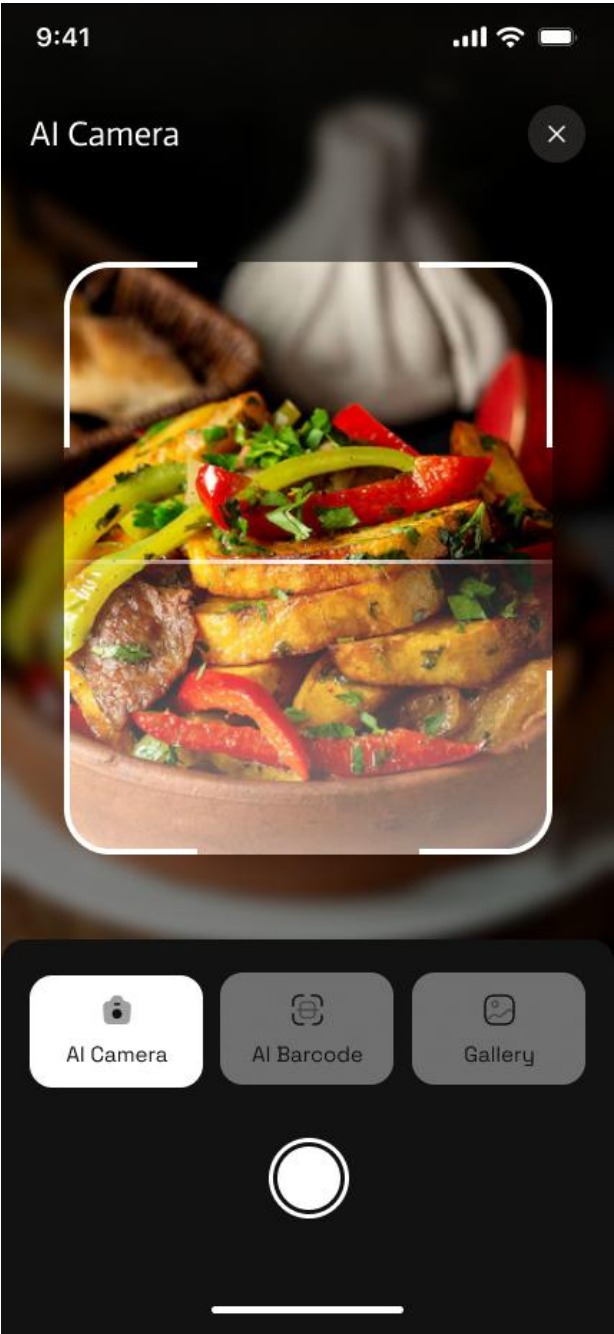


Figure 39: Scanning food screen

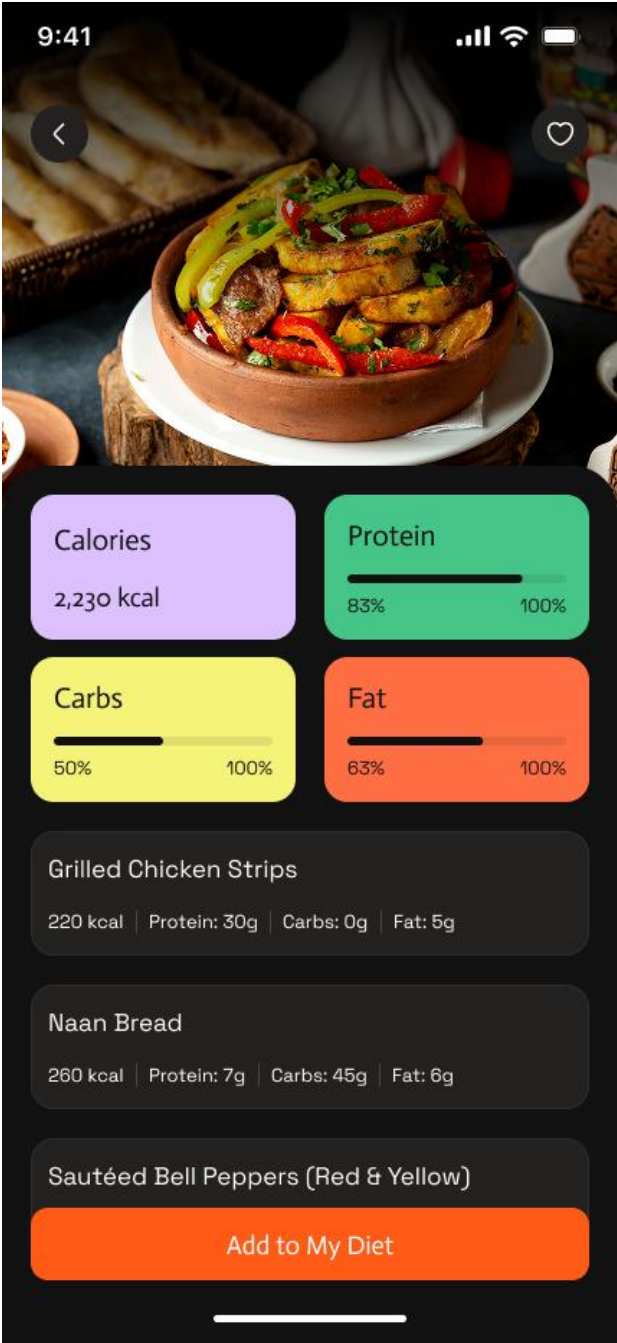


Figure 40: Scanning food Rresult Screen

5.5 Diet detail and Favorite diet screen



Figure 41: Diet detail Screen

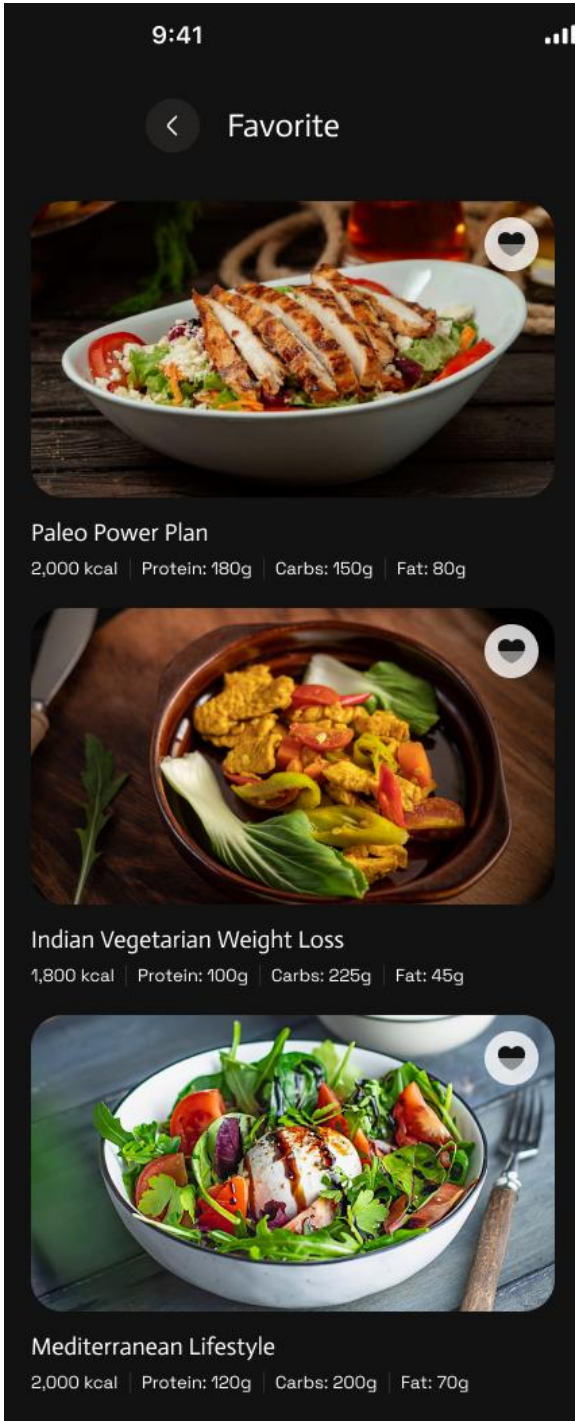


Figure 42: Favorite diet screen

5.6 Lucfin Chatbot UI: Start Screen and Message Screen

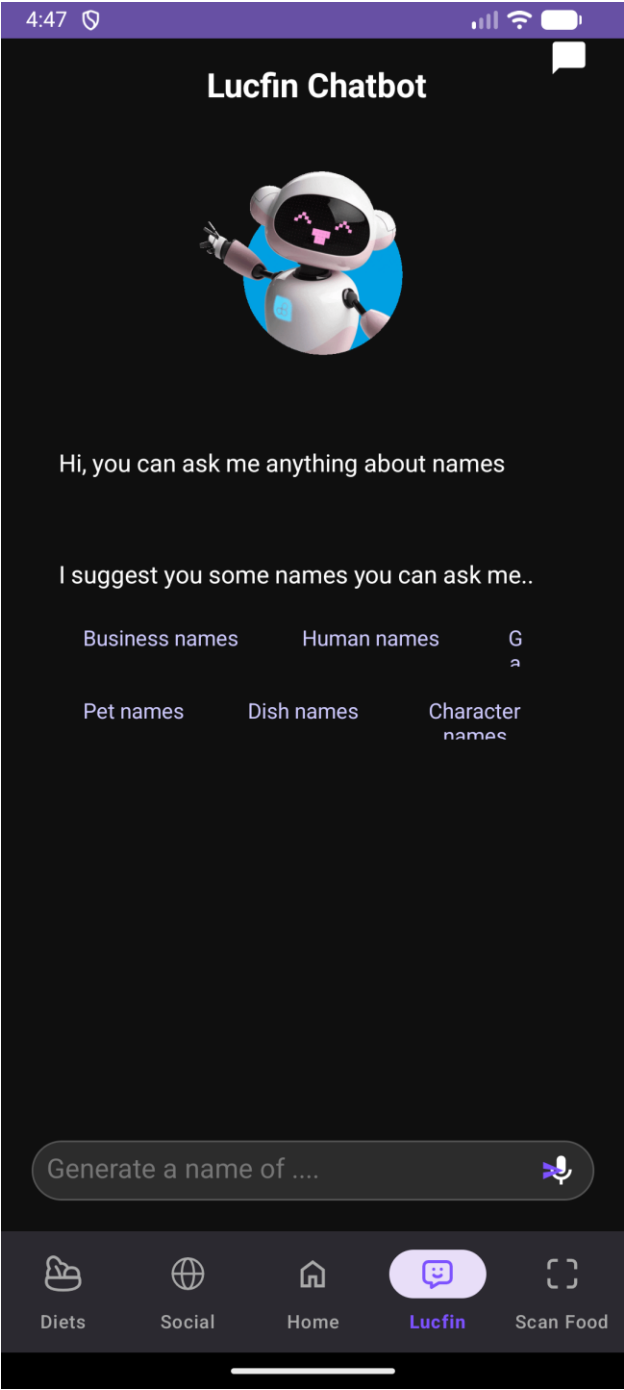


Figure 43: Start chat Screen

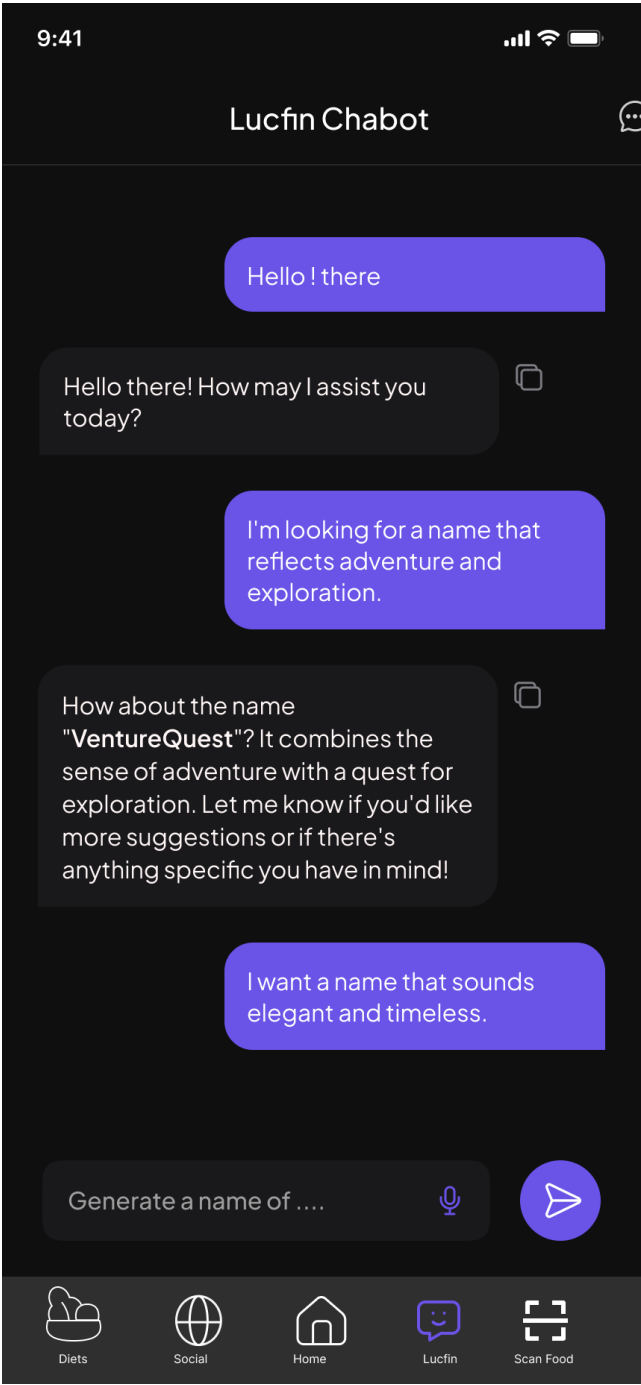


Figure 44: Message Screen

5.7 Lucfin Chatbot UI: Gen Answer and History Screen

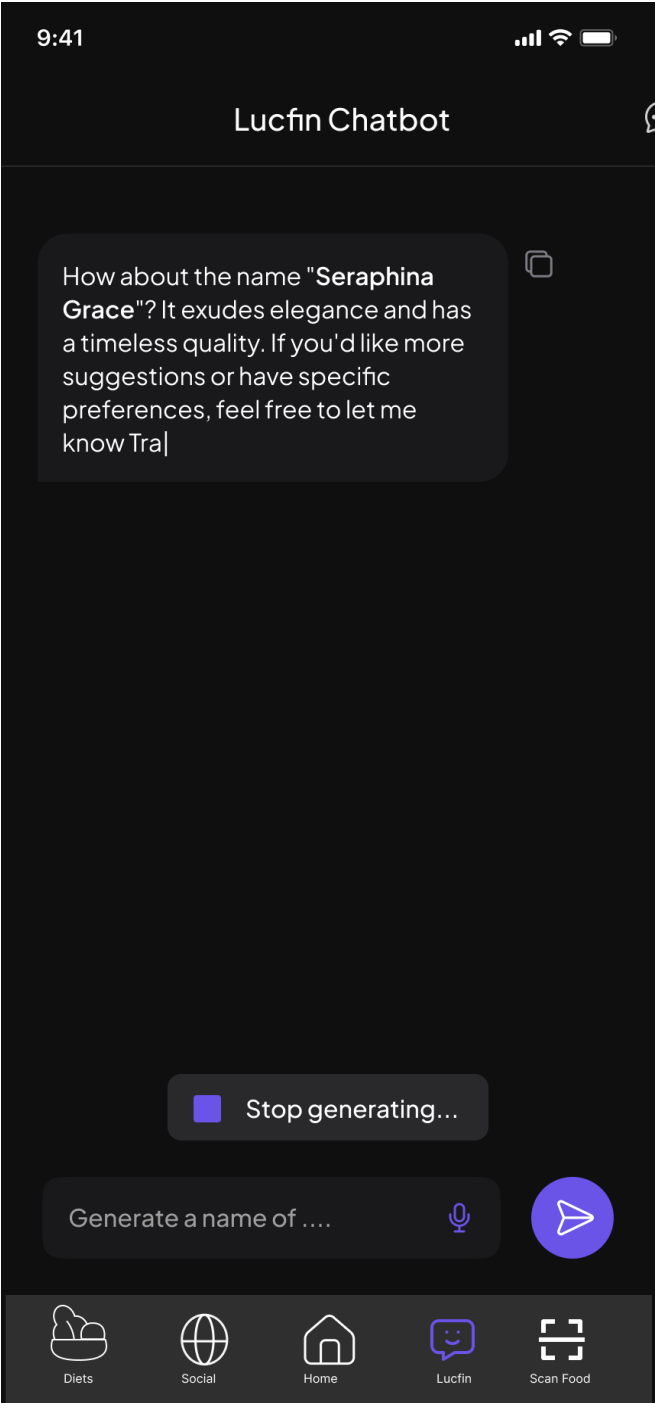


Figure 45: Gen Answer Screen

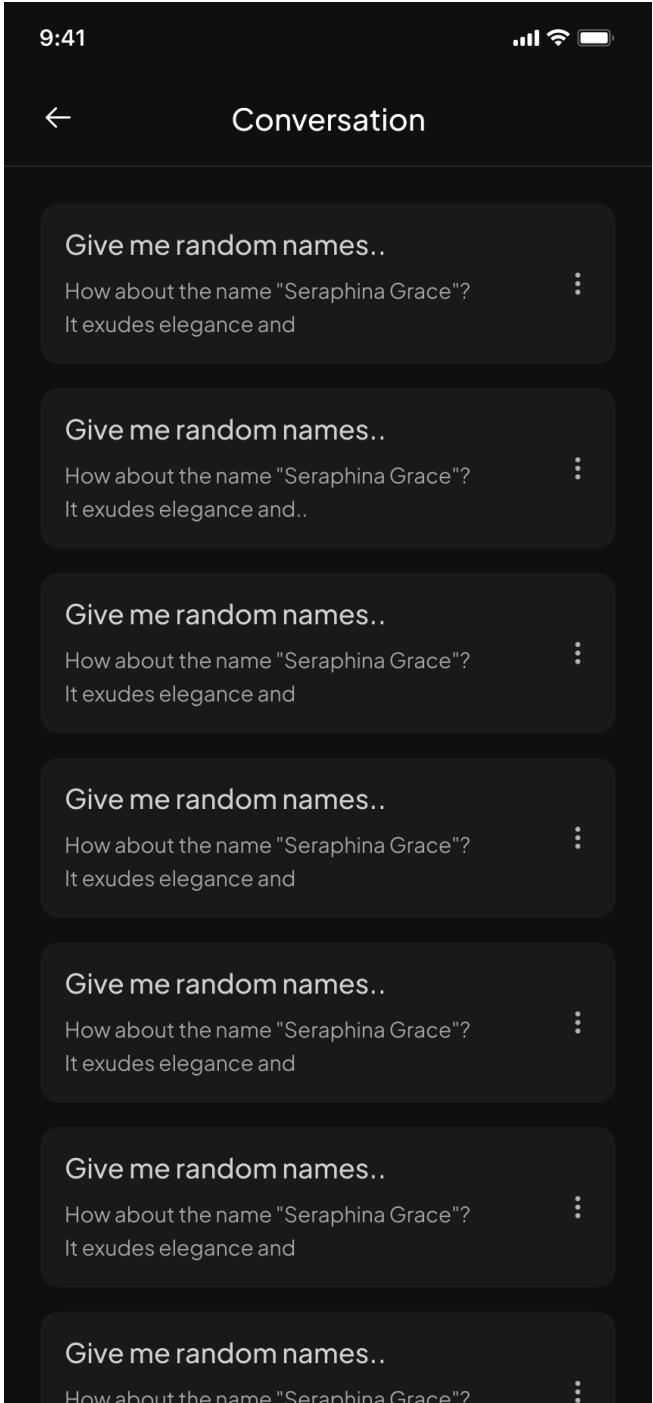


Figure 46: Chat History Screen

CHAPTER 5: FUTURE DEVELOPMENT DIRECTIONS

To ensure the system remains at the forefront of AI innovation, future development will pivot from a passive retrieval engine to a proactive **Autonomous Multi-Agent System**. Specifically, we aim to evolve the current RAG architecture into **Agentic RAG**, allowing the model to not just retrieve data but actively plan, reason, and execute multi-step workflows.

A core objective is implementing **Agent-to-Agent collaboration**. Specialized agents such as a "Vision Agent" utilizing advanced **Computer Vision** models (e.g., YOLO or ViT) for real-time image analysis, and a "Reasoning Agent" for logic processing will coordinate to solve complex, multimodal problems without human intervention. Additionally, we plan to integrate **GraphRAG** to enhance context understanding by mapping semantic relationships between data entities.

Finally, to support scalability, we will focus on **LLM Ops (MLOps)**. This involves optimizing inference latency via model quantization and deploying the solution on a containerized cloud infrastructure (Docker/Kubernetes), ensuring the application can handle high-concurrency real-time interactions efficiently.

CONCLUSION

This thesis has successfully designed and implemented Lucfin, an intelligent personal nutrition assistant that bridges the gap between static food databases and interactive health consultation. By synergizing Retrieval-Augmented Generation (RAG) with advanced Computer Vision, the system overcomes the limitations of traditional keyword search, delivering context-aware, evidence-based nutritional advice.

Crucially, the theoretical architecture has been materialized into a fully functional Android application. This deployment serves as empirical evidence of the system's viability, offering users a seamless experience that combines deep conversational intelligence with instant AI-powered food scanning. The application demonstrates that integrating LangChain orchestration with mobile-first deployment creates a robust solution for personalized dietary management. Lucfin not only solves the challenge of semantic information retrieval but also establishes a scalable foundation for future developments in AI-driven digital health solutions.

REFERENCES

- [1] Vaswani, A., et al. (2017). *Attention is All You Need*. *Advances in Neural Information Processing Systems*, 30
- [2] LlamaIndex Documentation: <https://docs.llamaindex.ai/en/stable/understanding/rag/>
- [3] Luong, Q. (2025). *Offline open-source RAG-based AI chatbot for Finnish-language software documentation on low-end hardware: Modular model-agnostic framework and AI judge for multilingual use*. Bachelor's Thesis, Degree Programme in Computer Applications.
- [4] Retrieval-Augmented Generation for Large Language Models: A Survey <https://arxiv.org/abs/2312.10997>
- [5] https://services.google.com/fh/files/misc/startup_technical_guide_ai_agents_final.pdf
- [6] <https://a2a-protocol.org/latest/>
- [7] <https://aclanthology.org/2024.paclic-1.4.pdf> A New Dataset and Empirical Evaluation for Vietnamese Food Recommendation System An Huynh-Quoc Tran, Hong Thi-Thuy Dang, Thanh Chi Dang, Tin Van Huynh Faculty of Information Science and Engineering, University of Information Technology, Ho Chi Minh City, Vietnam Vietnam National University, Ho Chi Minh City, Vietnam
- [8] https://huggingface.co/AITeamVN/Vietnamese_Embedding
- [9] <https://github.com/lannguyen0910/food-recognition>.
- [10] <https://epi.grants.cancer.gov/asa24/resources/portionsize.html>.
- [11] <https://arxiv.org/pdf/1506.01497> Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks